

VC-AWG

Technical report

2026

JANUARY 31

VNU - UET

Authored by:Hoang-Viet Tran



Contents

VC-AWG Technical report.....	3
System description	3
Use case list.....	3
Description of the codebase	4
Libraries and technologies used.....	4
Directory structure and system architecture	5
Detailed use case specification	10
Log in	10
Register	13
View the list of transactions.....	17
Create a new transaction	22
View list of bank accounts.....	27
Add a bank account.....	31
Edit account.....	35
Delete bank account	40
View bank account details	43
View Monthly Expenses	47
View Expenditure Details by Category	50
View Upcoming Invoices List.....	54
View the list of Goals.....	57
Create a New Goal.....	61
Adjusting Monthly Goals	64
View Savings Summary Chart.....	68
The Generated Website At Glance	72
Software Interface.....	72
Source code and automatically generated files	85

VC-AWG Technical report

This technical report presents the entire experimental process of the project, including a description of the initial system and source code, a detailed listing and analysis of input use case specifications, code generation results from coding prompts, corresponding exception scenarios, and an illustration of the generated software through interfaces and automatically generated file structures.

System description

The Financial Management system is a comprehensive personal finance management application designed to help users track cash flow, manage assets, analyze expenses, and set long-term financial goals. The system combines transaction data, account information, and analytical charts to provide a comprehensive view of an individual's financial situation. The main goal is to help users proactively control their finances, optimize spending habits, and achieve their savings targets.

The system's functional structure is organized into four core modules: account and balance management, transaction management, expense and billing analysis, and goal management. Each module corresponds to a group of use cases that directly serve the user's financial tracking and management needs.

Use case list

The system includes the following main use cases:

- **Account Registration:** Users create a new account by providing basic authentication information such as email, password, and necessary identification data.
- **Login:** This use case allows users to access the system by entering authentication information (email and password). The system verifies the validity of the login information and grants access if the information is correct.
- **View Transaction History:** Users can view all transactions, including transaction name, date, transaction amount, and transaction type.
- **Create a New Transaction:** This use case allows users to record a new transaction. When creating, users enter the amount, transaction type, and account. The system automatically updates the corresponding account balance.
- **View Bank Account List:** Users can view all accounts linked to the system, including bank name, account number, account type, and current balance.
- **Add bank account:** Users add a new account by providing the necessary information such as account name, bank, and initial balance. The system saves and updates the account list.
- **Edit Account:** This use case allows users to change information for an existing account, such as the display name or initial balance. The system records and updates the new data.

- **Delete Bank Account:** Users delete an account they no longer use. The system requires confirmation before proceeding and updates the account list after deletion.
- **View Bank Account Details:** This use case provides detailed information for a specific account, including an account description and a list of some recent transactions for users to track activity.
- **View Monthly Spending:** Users view a chart of total spending for the current month and previous months. The chart helps assess spending trends over time.
- **View Spending Details by Category:** This use case displays total spending grouped by category such as food, housing, transportation, etc. Users can easily see which category accounts for the largest proportion of spending this month.
- **View Bill List:** Users can view a list of recurring bills that are due soon. Each bill includes the name, amount, and due date to help users proactively manage payments.
- **View Monthly Goals:** The system displays a list of existing financial goals, including savings goals or spending limits by category. Users can track the completion status of each goal.
- **Create New Goal:** Users create new financial goals by entering the goal value and applicable category. The system saves and adds them to the list of monitored goals.
- **Adjust Monthly Goals:** The use case allows users to change the goal value, update the amount achieved, or edit the scope of application. The system updates and reflects the new progress.
- **View Savings Overview Chart:** Users can view a chart comparing net savings by month in the current year and the previous year. The chart helps assess the level of financial improvement over time.

Description of the codebase

Libraries and technologies used

The project employs a modern and integrated set of technologies, focusing on the TypeScript/Node.js ecosystem to optimize performance and code maintainability.

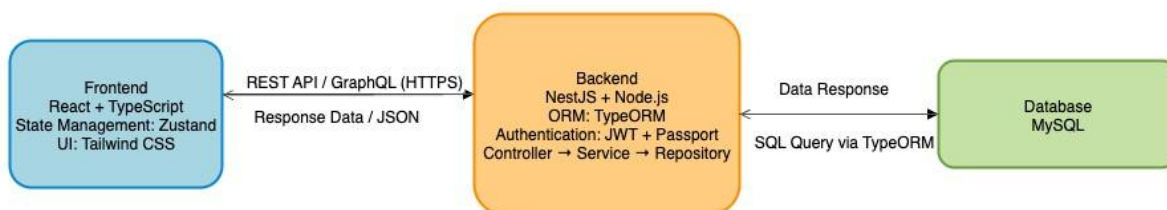


Figure 1. The overall architecture diagram of the system: Backend - Frontend - Database.

The backend is built using NestJS on the Node.js platform. NestJS provides a powerful, tightly structured framework that supports the Module/Controller/Service model and applies object-oriented programming (OOP) design patterns. This makes the backend architecture easily scalable and scalable,

similar to traditional frameworks like Spring Boot. To interact with the database, TypeORM is used in conjunction with MySQL, acting as the Object-Relational Mapping (ORM) layer, allowing for the definition of Entities to manipulate the database securely and clearly. Authentication is performed via JWT (JSON Web Token), integrated through the @nestjs/jwt module and the Passport strategy.

The frontend is developed using the React library (using TypeScript), adhering to a Component-Based model, enabling interface reuse and efficient UI management. Vite was chosen as the bundler due to its rapid development speed and optimization capabilities. Complex state management is handled by the Zustand library, known for its simplicity, ease of understanding, and high performance. The user interface is styled using Tailwind CSS, a utility-first CSS framework that allows developers to quickly build custom designs without writing extensive custom CSS.

Directory structure and system architecture

The system's directory structure is clearly divided into two root directories, representing the physical separation between the two independent running applications:

- /be: Contains the complete Backend (NestJS) source code..
- /fe: Contains the complete Frontend source code (React + Vite).

Detailed Backend Structure (NestJS): The NestJS backend adheres to a modular model and organizes its source code using a feature-based architecture. The main components in the /be/src directory are arranged as follows:

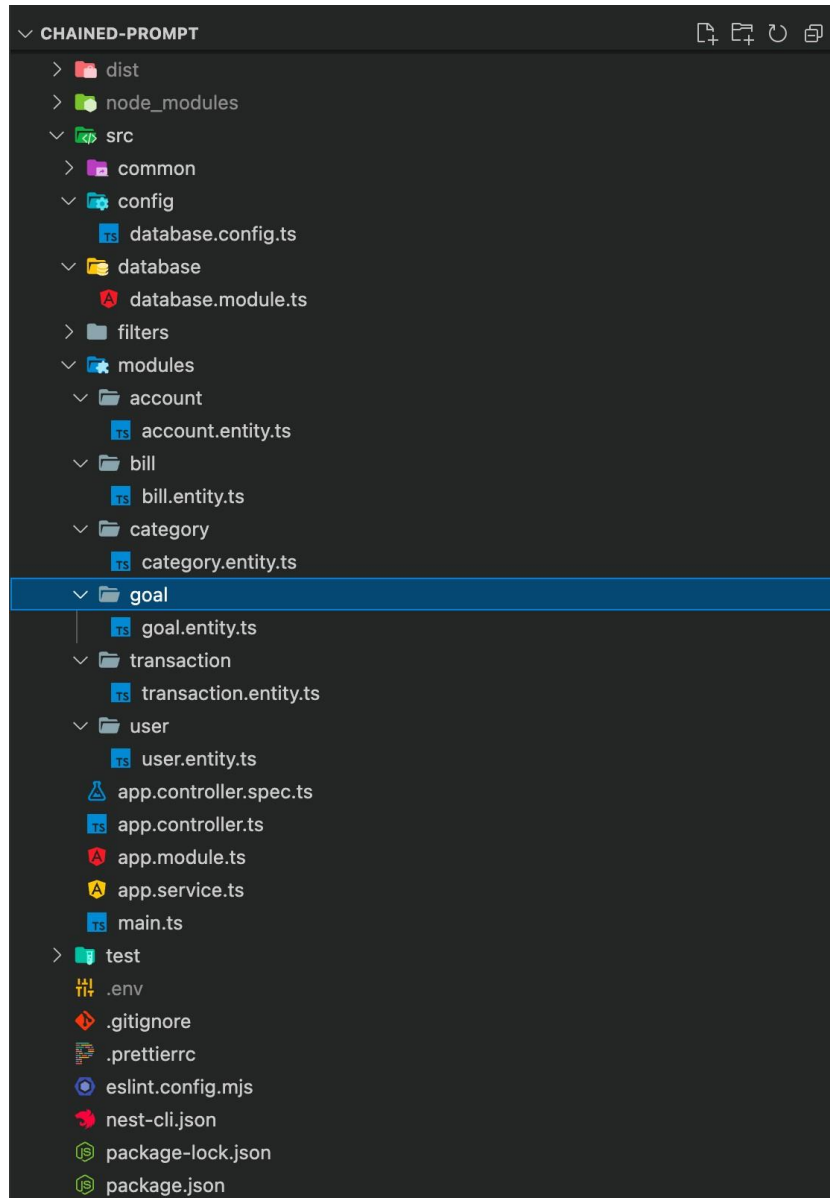


Figure 2. Backend directory structure of the project.

- `main.ts`: Application startup file; CORS, Swagger, and Global Validation Pipe configuration..
- `app.module.ts`: The base module integrates various business modules; it requires importing the `DatabaseModule`.
- `config/`: Contains configurations unrelated to business requirements.
 - `database.config.ts`: Configure MySQL connection using environment variables.
- `database/`: Contains ORM connection components.
 - `database.module.ts`: Configure TypeORM to connect to MySQL, `synchronize: false`.

-
- `common/`: Contains shared Guard, Interceptor, and Pipe; where JWT Strategy and Validation Pipes are implemented..
 - `filters/`: Contains a formatter and a global error filter.
 - `http-exception.filter.ts`: HTTP error filtering (needs further improvement).
 - `modules/`: It contains all business logic modules such as user, account, transaction, etc. Each module includes: Controller, Service, Entity, and DTO..
 - `entities/`: Define shared entities. This includes the classes User, Account, Transaction, Bill, Category, and Goal..

Detailed Frontend Structure (React + Vite): The React frontend is designed using a component architecture with combined centralized state management. The main directories in `/fe/src` include:

- `api/`: The layer that communicates with the Backend API..
 - `axiosInstance`: Base URL, Request Interceptor (automatically attaches JWT), and Response Interceptor (handles 401/logout) have been configured..
- `components/`: Reusable UI elements include Button, Input, Layout, Loading, Error, and Navigation Bar.
- `pages/`: The main pages of the application include: Home, Login, Register, Dashboard, and business pages. Lazy Loading is applied..
- `context/`: Managing global state via React Context..
 - `AuthContext`: Manage login status and tokens..
 - `ThemeContext`: Theme management.

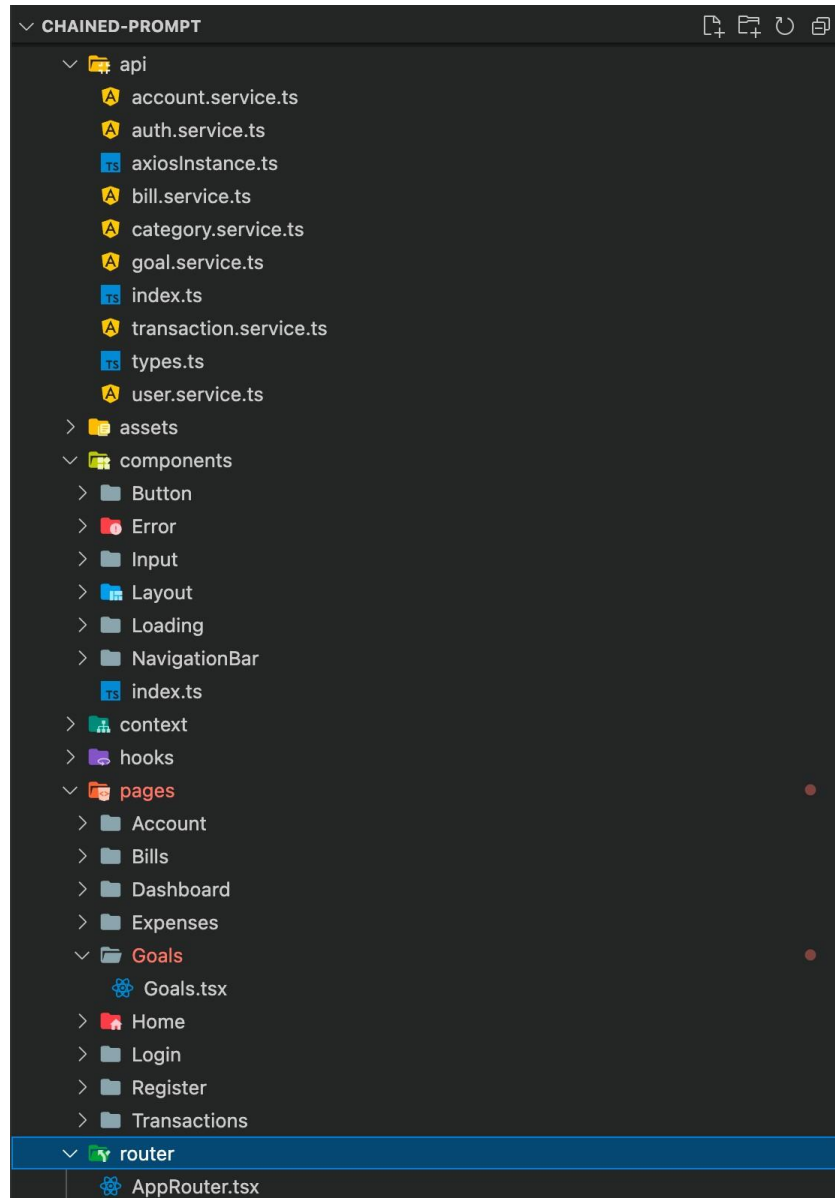


Figure 3. Describe the Frontend directory structure of the project.

- router/: Defining a routing system.
- ProtectedRoute: Protect routes that require login..
- hooks/: Contains custom hooks such as useDebounce and useLocalStorage.
- types/: Define TypeScript interfaces such as User, Account, Transaction, and ApiResponse<T>.

Main components:

To ensure consistency, security, and performance, the project established key fixed components right from the initial phase:

Backend (NestJS): The following platform configurations are set in the main startup function (main.ts) and the corresponding configuration files:

- **Database Configuration:** A TypeORM connection to MySQL is established, using environment variables to extract login information. A crucial configuration is ``synchronize: false`` to ensure the stability and security of the existing data schema
- **Swagger API Documentation:** API documentation is automatically generated and served at the ``/api/docs`` endpoint. This configuration integrates Bearer Auth (JWT), allowing testing of authentication-required APIs directly on the Swagger interface.
- **CORS (Cross-Origin Resource Sharing):** This configuration is extended to allow API access from multiple sources (client frontend) using values from environment variables or defaults. It fully supports essential HTTP methods (GET, POST, PUT, DELETE, PATCH) and allows sending credentials.
- **Entities:** The core entities of the financial management system have been defined (User, Account, Transaction, Bill, Category, Goal), acting as the interface between business logic and the database.
- **Exception Filter Structure:** The `HttpExceptionFilter` file has been created, acting as a global exception handling mechanism, ensuring that all error responses sent to the client adhere to a consistent JSON format (`"success": false, "message": "..."`).

Frontend (React + Vite): The fixed components in the frontend focus on standardizing API communication and managing authentication state:

- **Axios Instance:** A custom instance of Axios has been set up. It includes:
 - Base URL is fixed to `/api` via Vite's proxy, ensuring API path consistency.
 - Request Interceptor: Automatically inserts the JWT Token (obtained from authentication status) into the `Authorization:Bearer <token>` header of every outgoing request.
 - Response Interceptor: Automatically handles HTTP 401 (Unauthorized) error responses, triggering the logout process and redirecting the user to the login page.
- **Routing System:** The routing system (React Router DOM) is clearly divided into Public Routes and Protected Routes. The `ProtectedRoute` component is used to protect business pages, based on authentication status from `AuthContext`.
- **Context APIs:**
 - *AuthContext:* This is the central management system for user authentication status, including login status and storage/provision of JWT tokens.
 - *ThemeContext:* Manages the user interface theme, ensuring a consistent user experience.

- **API Services Structure:** The structure of the pre-created API service files (auth.service.ts, user.service.ts, etc.) acts as an abstraction layer between the user interface and the backend endpoints.

Project code generation rules

The project applies standard naming conventions and standardized Git workflows. specifically, all API communication adheres to the standard JSON response format. All business logic in the automated code generation process must comply with the principles of "Do not place business logic in Controller/Component" and "Create all 5 components (controller, service, entity, dto, module) for each new NestJS module."

The backend architecture strictly adheres to a modular model and task separation:

- Business logic must reside within the Service. The Controller only handles requests and calls to the Service.
- Use the TypeORM Repository pattern to access the database.
- DTOs are used in conjunction with class-validators to check data validity.

The frontend adopts a Component-Based architecture with the following key rules: Use Functional Components and Hooks for development; the interface is styled using Tailwind CSS; state is managed using Context APIs (Auth, Theme) or Zustand (business data); API communication always goes through the Axios Instance in the src/api/ directory to leverage Interceptors .

Detailed use case specification

Within the scope of this section, the thesis only presents a few typical use cases to clearly illustrate how to apply the proposed method. The remaining use cases, due to structural overlap or not significantly affecting the understanding of the overall process, will be included in the Appendix to avoid making the document too lengthy.

Log in

Use Case Specification

Table 1 presents the Login use case specification, which allows users to access the system with valid credentials.

Table 1. System Login Use Case Specification

Use Case Name	Login
Description	Allows the user to authenticate their identity using email and password to access the system. Supports the "Save login information" option.

Actor	User
Main Flow	The user accesses the Login button on the Homepage. The system redirects to the Login page. The user enters their email and password. Clicks the "Login" button. The system checks for validity. If valid: establish a login session. Redirects to the Homepage and displays "Logout".
Exception Flow	If the email or password is incorrect: displays the error "Incorrect email or password." The user remains on the Login screen.
Alternative Flow	The user selects "Save login information". If successful, the system remembers the login session.
Precondition	The user is on the Homepage. A valid account is already present.
Postcondition	Success: Login and redirects to the Homepage. Failure: An error is displayed.

UI design in Figma

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=137-7477&t=cxgXQfkGoLMfxW48-4>

API Contract

Endpoint:

POST /api/auth/login

Request Body:

```
{ "email": "johndoe@email.com",
```

```
"password": "mypassword" } Response upon success:
```

```
{"accessToken": "JWT_TOKEN",
```

```
"user": {"id": 1, "fullName": "John Doe", "email": "johndoe@email.com"} }
```

Error response:

```
{ "error": "Incorrect email or password." }
```

Coding Prompt

1. Prompt A: Backend code generation (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling for the login functionality..

Create a POST endpoint `api/auth/login` in NestJS's `AuthController`. This endpoint is a public route and does not require Guard authentication.

Request format:

```
{ "email": "string",  
  "password": "string" }
```

AuthService processing logic:

Receive email and password from the request body.

Find the user in the database using the email.

If not found or the password does not match, processing fails.

If correct, create a JSON Web Token (JWT) containing the user's identifier.

Return the token and the user's basic information.

Successful Response format:

```
{ "accessToken": "JWT_TOKEN",  
  "user": { "id": 1, "fullName": "John Doe", "email": "johndoe@email.com" } }
```

Error Handling: If the email does not exist or the password is incorrect, the system throws an `UnauthorizedException` with the following JSON error:

```
{ "error": "Incorrect email or password." }
```

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: Build the login interface according to Figma's design.

Create the `LoginForm` component using React TypeScript and Tailwind CSS. The form includes: Email input field, Password input field, "Save login information" checkbox, "Login" button, "Forgot password?" link.

Please adhere strictly to the Figma MCP design at the following link:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=137-7477&t=Jwl8SfgcguZ5Gicc-4>

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI form to the API and handle the successful login flow.

Add states: email: string, password: string, rememberMe: boolean. Implement the asynchronous handleSubmit function to send a POST request to api/auth/login.

Payload sent:

```
{"email": "johndoe@email.com",  
  "password": "mypassword"}
```

Upon successful login:

Save the accessToken to localStorage.

Save user information to the global state (Context API / Redux / Zustand).

Redirect the user to the homepage /.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve client-side error handling, loading, and validation.

Upgrade the handleSubmit function with the following features:

Loading State: Add the state `isLoading`. While processing, disable the Login button and display a spinner.

API Error Handling: If the API returns a 401 error, display the message: "Incorrect email or password."

FE Validation:

Email cannot be blank and must be in the correct format.

Password cannot be blank.

If an error occurs, display a message directly below the input field.

Register

Use Case Specification

Table 2 details the Account Registration use case, which allows users to create a new account by providing authentication information.

Table 2. Use Case Specification: Account Registration

Use case name	Register
Description	Allow new users to provide information (Full name, Email, Password) to create a new account in the system.
Actor	User

Main flow	Users access the Homepage and click the Register/Login button. The system redirects you to the Registration page. The user enters their full name, email address, password, and confirm password. Press the "Sign Up" button. The system checks the validity of the input data. Does the email verification system already exist? Password verification and confirmation system. If valid: create a new account. Set up a login session for new users. Redirect to Homepage (/).
Alternative flow	A.1 – Email already exists After step 6, the main flow. If the email address is already registered: the error message "This email is already registered" will be displayed. The user remains on the Registration screen. A.2 – Password does not match After step 7, the main flow. If they don't match: display the error "Passwords do not match." The user remains on the Registration screen. A.3 – Invalid data If the email is incorrectly formatted or a field is left blank. The system displays the error message directly below the input field. The user remains on the Registration screen.
Preconditions	The user is currently accessing the Registration screen. The user does not yet have an account in the system.

Postconditions	Success: Account created, login session established, and redirected to Homepage (/). Failure: An error is displayed and the user remains on the Registration screen.
----------------	--

UI design in Figma

<https://www.figma.com/design/iEOnfperOrck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=137-8071&t=cxgXQfkGoLMfxW48-4>

API Contract

Endpoint:

POST /api/auth/register

Request Body:

```
{ "fullName": "Nguyen Van A", "email": "example@email.com", "password": "123456",
  "confirmPassword": "123456" }
```

Response upon success:

```
{ "message": "Registration successful", "user": { "id": 1, "fullName": "Nguyen Van A", "email":
  "example@email.com" },
  "token": "JWT_TOKEN" }
```

Error response:

```
{ "error": "This email is already registered." }
{ "error": "Passwords do not match." }
```

Coding Prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build an API service, business logic, and server error handling for the account registration function.

Create a POST endpoint `api/auth/register` in NestJS's `AuthController`. This endpoint is *a public route* and does not require Guard.

Request format:

```
{ "fullName": "Nguyen Van A",
  "email": "example@email.com", "password": "123456",
  "confirmPassword": "123456" }
```

Processing logic in AuthService:

Use DTOs and class-validators to check the following fields: fullName, email, password.

Check if the password matches confirmPassword.

Check if the email address already exists in the database.

If valid, hash the password and create a new user.

Generate JWT tokens for new users.

Returns user information and token.

Response formatting successful:

```
{ "message": "Registration successful", "user": { "id": 1, "fullName": "Nguyen Van A", "email": "example@email.com" },  
  "token": "JWT_TOKEN" }
```

Error handling:

```
{ "error": "This email is already registered." }
```

```
{ "error": "Passwords do not match." }
```

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build a registration interface that matches the Figma design.

Create a SignUpForm component using React TypeScript and Tailwind CSS. The form should include: Email, Full Name, Password, Confirm Password, a *Sign Up button*, and a link to the login page.

The design must closely follow Figma MCP at the following link:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=137-8071&t=cxgXQfkGoLMfxW48-4>

3. Prompt C: Generate Frontend Logic Code (State & API Connection)

Objective: Connect the UI form to the API and handle successful registration.

Add the following states: fullName, email, password, confirmPassword.

Implement the handleSignUp function to send a POST request to api/auth/register.

Payload uploaded:

```
{ "fullName": "...",  
  "email": "...",  
  "password": "...",  
  "confirmPassword": "..." }
```

Upon successful registration:

Save the token to LocalStorage or Context.

Save user information to global state.

Redirect users to the homepage /.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To complete client-side error handling, loading, and validation.

Loading: Add the state `isLoading`. While processing, disable the Sign Up button and display the spinner.

API error handling: Display the error from the response, such as: *"This email is already registered."*

"Passwords do not match."

Validation FE:

Check that all fields are not left blank.

Check the password and confirm that it matches.

If an error occurs, display a message directly below the input.

View the list of transactions

Use case specification

Table 3 describes the use case "View transaction list," which allows the user to track all completed transactions.

Table 3. Use case specification: View transaction list

Use case name	View the list of transactions.
Description	Allows users to view a list of completed transactions (Income and Expenses). Display the following information: item name, store, transaction date, payment method, and amount.
Actor	User

Main flow	Users access the Transactions page via the sidebar (/transactions). The system displays the "All" tab by default. The front-end calls the API. The backend verifies the access_token and retrieves the user_id. The backend queries transactions by user_id, sorts them in descending order by date, and applies pagination. The backend returns a list of transactions. The front-end displays a list with the following columns: Items, Shop Name, Date, Payment Method, and Amount.
Exception flow	Authentication or query error: Backend returned error (401, 500). The front-end displays an error message and fails to load the list.
Alternative flow	Filter by Income/Expenses: Users select Revenue or Expenses, and the system returns a filtered list. Loading more data: Users scroll down or select "Load More", the system increases the offset and loads more transactions. No transactions recorded: If the list is empty, display "You have no transactions recorded."
Preconditions	The user is logged in. The user has made at least one transaction.
Postconditions	Success: The list of transactions is displayed (can be filtered or paginated). Failure: Displays an error message or no data.

UI design in Figma

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=66-5474>

---Financial-Management-

API Contract

Endpoint:

GET /api/v1/transactions

Query Parameters:

type: All, Revenue, or Expense

limit: Number of records to retrieve at each time (default 10)

Offset: Starting position (default 0, serving paging) Response success (200):

```
{"data": [  
  {"transactionId": 1,  
    "accountId": 3,  
    "transactionDate": "2023-05-17T00:00:00Z",  
    "type": "Expense",  
    "itemDescription": "GTR 5",  
    "shopName": "Gadget & Gear",  
    "amount": 160.00  
    "paymentMethod": "Credit Card",  
    "status": "Complete",  
    "receiptId": null,  
    "createdAt": "2023-05-17T08:12:00Z"},  
  {"transactionId": 2,  
    "accountId": 3,  
    "transactionDate": "2023-05-17T00:00:00Z",  
    "type": "Expense",  
    "itemDescription": "Polo Shirt",  
    "shopName": "XL Fashions",  
    "amount": 20.00  
    "paymentMethod": "Credit Card",  
    "status": "Complete",  
    "receiptId": null,  
    "createdAt": "2023-05-17T08:12:00Z"}],  
  "total": 2  
  "hasMore": false}
```

Response error (400 / 500):

```
{ "message": "Invalid type parameter" }
```

Coding Prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build an API service, business logic, and server error handling for the transaction list retrieval function.

Create a GET endpoint at `/api/v1/transactions` in `TransactionsController`. This endpoint must be protected by `JwtAuthGuard`.

Request format:

type: "All""Revenue""Expense" (required)

limit: number (default: 10)

offset: number (default: 0)

Processing logic in `TransactionsService`:

Get the `user_id` from the authenticated request.

Receive query parameters: type, limit, offset.

Check that the type is one of: "All", "Revenue", "Expense".

Build a query to retrieve transactions based on `user_id`.

If the type is "Revenue" or "Expense", add a filtering condition.

Sort by `transactionDate` in descending order.

Apply paging: limit, offset.

Returns a list of transactions, the total number of records, and the `hasMore` value.

Response formatting successful:

```
{"data": [  
  {"transactionId": 1,  
    "accountId": 3,  
    "transactionDate": "2023-05-17T00:00:00Z",  
    "type": "Expense",  
    "itemDescription": "GTR 5",  
    "shopName": "Gadget & Gear",  
    "amount": 160.00
```

```
"paymentMethod": "Credit Card",
"status": "Complete",
"receiptId": null,
"createdAt": "2023-05-17T08:12:00Z"}],
"total": 1,
"hasMore": false}
```

Error handling:

```
{ "message": "Invalid type parameter" }
```

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build a transaction list interface that matches the Figma design.

Create TransactionsPage component using React TypeScript and Tailwind CSS.

The interface must include:

Title of the Transactions page.

Three tabs: All, Revenue, Expense (the selected tab must have a prominent style).

The table or list displays transactions with the following columns: Items, Shop Name, Date, Payment Method, Amount.

Icons illustrate each transaction.

Load More button (displayed when there is still data).

The message is empty: "You have no transactions yet."

The design must match the Figma MCP template 100%:
<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=66-5474> ---Financial-Management-

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and handle the transaction data reception flow.

Add states to the TransactionsPage:

```
transactions: Transaction[]
```

```
activeTab: 'All' | 'Revenue' | 'Expense'
```

```
pagination: { offset, limit, hasMore }
```

```
isLoading: boolean
```

```
error: string | null
```

Implement the `fetchTransactions(loadMore = false)` function to call the API:

If loadMore = false: offset = 0.

If loadMore = true: offset = pagination.offset.

Response format:

```
{"data": [...],  
"total": 2  
"hasMore": false}
```

Processing successful:

If loadMore = true: append the data to the old array.

If false: replace the entire list.

Update pagination.

Call the API whenever the activeTab changes.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling, loading, and list status display.

Loading: When calling the API, enable isLoading. While loading, display *Spinner/Skeleton* instead of a list.

API Error: In the catch block, set error = "Could not load transaction list. Please try again." Display the error instead of the list.

No data: If the transactions tab is empty and not loading, display: *"You have no transactions for the activeTab"*.

Validation FE: Not applicable as there is no data entry form.

Create a new transaction

Use case specification

Table 4 details the use case for creating a new transaction, allowing the user to record a transaction and update the account balance.

Table 4. Use case specification: Create new transaction

Use case name	Add a new transaction
---------------	-----------------------

Description	Allows users to record a new financial transaction (income or expenses) by providing necessary information such as transaction details, amount, transaction type, and category.
Actor	User
Main flow	Users access the Transactions page and click the “Add transaction” button. The system displays an input form with the following fields: Transaction Name, Item Name, Category, Amount, Transaction Type (Revenue / Expense), Payment Account, Transaction Date, and other options (Payment Method / Related Account). Users fill in all the required information on the form. The user clicks the "Save" button. The front-end system sends a request with data to the backend API. The backend system verifies the access_token and extracts the user_id. The backend system checks the validity of the input data. (Transaction name must not be blank, Amount > 0, Transaction type is valid, Category exists, Account belongs to user_id). If valid, save the new transaction to the Transactions table and update the related account balance. The backend returns a success response. The front-end receives the response and displays the message "Transaction added successfully." The front-end refreshes the transaction list or redirects back to the /transactions page.

Alternative flow	A.1 – Invalid data After step 7, the main flow. If data is missing or incorrectly formatted, display the corresponding error next to the input field. The user remains on the Add Transaction form. A.2 – Invalid account If the payment account does not belong to user_id or the balance is insufficient: display an error message. • The user remains on the Add Transaction form.
Preconditions	Users must be logged in. Categories must be available for users to choose from.
Postconditions	• Success: The new transaction is saved to the Transactions table, a success message is displayed, and the transaction list is updated. • Failure: The system displays an error message corresponding to invalid data, and the user remains on the Add Transaction form.

UI design in Figma

The add transaction form has a layout similar to the overall style of the Transactions page (using rounded corners, input fields with faint borders, and a blue Save button). The fields are arranged vertically and have clear labels. The Cancel button returns to the /transactions page.

API Contract

Endpoint:

POST /api/v1/transactions

Request Body:

```
{"accountId": 3, "transactionDate": "2023-05-17T00:00:00Z", "type": "Expense", "itemDescription": "Movie Ticket", "category_id": 3, "shopName": "Inox", "amount": 150000, "paymentMethod": "Credit Card", "status": "Complete"}
```

Response successful (201 Created):

```
{"message": "Transaction created successfully",  
"data": {"transactionId": 8, "accountId": 3, "transactionDate": "2023-05-17T00:00:00Z", "type": "Expense", "itemDescription": "Movie Ticket", "shopName": "Inox", "amount": 150000,
```

```
"paymentMethod": "Credit Card", "status": "Complete", "receiptId": null, "createdAt": "2023-05-17T08:12:00Z", "category_id": 3}}
```

Response error (400 / 422):

```
{"message": "Invalid or missing transaction data"}
```

Coding Prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling for the transaction adding functionality.

Create a POST endpoint `/api/v1/transactions` in the NestJS `TransactionsController`. This endpoint must be protected by `JwtAuthGuard`.

Request format:

```
{"accountId": 3,  
"transactionDate": "2023-05-17T00:00:00Z",  
"type": "Expense",  
"itemDescription": "Movie Ticket", "category_id": 3,  
"shopName": "Inox",  
"amount": 150000  
"paymentMethod": "Credit Card",  
"status": "Complete"}
```

Processing logic in `TransactionsService`:

Receive transaction data (DTO) and `user_id` from `JwtAuthGuard`.

Check the validity of the data:

`itemDescription` cannot be empty.

The amount must be greater than 0.

The type must be 'Revenue' or 'Expense'.

The `category_id` must exist and be owned by the user.

Check if `accountId` belongs to `user_id`.

Check your account balance if the transaction type is 'Expense'.

Perform transactions in the database:

Save the new transaction to the `Transactions` table.

Update the balances in the Accounts table by transaction type.

Commit the transaction if successful.

Response format successful (201 Created):

```
{"message": "Transaction created successfully",  
"data": {  
  "transactionId": 8,  
  "accountId": 3,  
  "transactionDate": "2023-05-17T00:00:00Z",  
  "type": "Expense",  
  "itemDescription": "Movie Ticket",  
  "shopName": "Inox",  
  "amount": 150000  
  "paymentMethod": "Credit Card",  
  "status": "Complete",  
  "receiptId": null,  
  "createdAt": "2023-05-17T08:12:00Z",  
  "category_id": 3}}
```

Error handling: If the data is invalid (e.g., amount ≤ 0 , accountId does not exist), throw a `BadRequestException` with JSON:

```
{"message": "Invalid or missing transaction data", "statusCode": 400}
```

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build a user interface for adding new transactions based on the Figma design.

Create `AddTransactionForm` component using React TypeScript and Tailwind CSS.

The interface includes:

Input for 'Transaction Name' (Transaction Name / Item Description)

Dropdown/Select options for 'Category'

Input a number for the 'Amount'

Radio button or Select for 'Transaction Type' (Type: Revenue / Expense)

Dropdown/Select for 'Payment Account'

Input date type for 'Transaction Date'

The blue 'Save' button

The 'Cancel' button will take you back to the previous page.

Strict design according to UI Notes: rounded card corners, faint input border, vertical field layout, clear labels.

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

In AddTransactionForm, add state for the fields:

itemDescription, amount, categoryId, type, accountId, transactionDate, paymentMethod.

Implement the asynchronous handleSubmit function to send requests to the POST address in /api/v1/transactions.

Payload request:

```
{"accountId": 3,  
  "transactionDate": "2023-05-17T00:00:00Z",  
  "type": "Expense",  
  "itemDescription": "Movie Ticket", "category_id": 3, "amount": 150000}
```

Actions to take after success:

Display toast: "Transaction added successfully.", Reset form, Navigate to the /transactions page.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling, loading, and client-side validation.

Loading Status: Before calling the API, set isLoading = true. While loading, disable the 'Save' button and display the spinner. Set isLoading = false after the API is complete.

API Error: Use try...catch to catch API errors. If it returns 400/422/500, display the toast: "Cannot add transaction at this time. Please try again later."

Validation FE: Before calling the API, check: 'Transaction Name' must not be empty, 'Amount' > 0, 'Category' and 'Payment Account' must be selected.

If validation fails, display an error message directly below the input and do not call the API.

View list of bank accounts

Use case specification

Table 5 describes the use case "View list of bank accounts," which displays all linked accounts of the user.

Table 5. Use case specification: View list of bank accounts

Use case name	View a list of bank accounts.
Description	Allows logged-in users to access and view a list of all bank accounts or e-wallets they have linked to the system, including details such as balances and account types.
Actor	User
Main flow	The user is logged in and has a valid access token. Users access the "Bank Account" page. The front-end system calls the back-end API. The backend verifies the access_token and extracts the user_id. The backend queries the Accounts table based on user_id. The backend returns a list of accounts (Bank, Account Type, Branch, Account Number, Balance). The front-end receives the data and displays a detailed list of accounts.
Alternative flow	A.1. No linked account: If the returned list is empty: display the message "You haven't linked any accounts. Please add an account," and provide a button to add a new account. A.2. Authentication Error / Access Error: If the access_token is invalid or expired: return an HTTP 401 error, redirect the user to the Login page, and display a message requesting them to log in again. A.3. Backend System Errors: If a database query or internal processing error occurs: display a friendly error message "A system error has occurred, please try again later."

Preconditions	Users must be logged in (possessing a valid access token). The user has linked at least one bank account/e-wallet.
Postconditions	• Success: The system displays a complete list of the user's accounts on the "Bank Accounts" page. • Failure: The system displays an error message (e.g., access error, no data) and the user remains on the "Bank Account" page or is redirected to the error message page.

UI design in Figma

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5320&t=cxgXQfkGoLMfxW48-4>

API Contract

Endpoint:

GET /api/v1/accounts

Request:

Authorization: Bearer <access_token>

Sample response:

```
{"user_id": 1, "accounts": [ { "id": 1, "bank_name": "Vietcombank", "account_type": "Checking",
"branch_name": "District 1", "account_number_last_4": "0123", "balance": 4000000}, { "id": 2,
"bank_name": "Techcombank", "account_type": "Savings", "branch_name": "District 3",
"account_number_last_4": "0987", "balance": 4000000}]}
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build an API service, business logic, and server error handling for the function of retrieving a list of accounts.

Create a GET endpoint /api/v1/accounts in NestJS AccountsController. This endpoint must be protected by *JwtAuthGuard* .

Request format: No request body. User information is retrieved from access_token.

Processing logic in AccountsService:

Get user_id from JWT payload.

Query the database to find all accounts with the corresponding user_id.

Returns a list of accounts. If no account is found, returns an empty array.

Successful response:

```
{"user_id": 1,
"accounts": [
  { "id": 1,
    "bank_name": "Vietcombank",
    "account_type": "Checking",
    "branch_name": "District 1",
    "account_number_last_4": "0123",
    "balance": 4000000},
  {"id": 2,
    "bank_name": "Techcombank",
    "account_type": "Savings",
    "branch_name": "District 3",
    "account_number_last_4": "0987",
    "balance": 4000000}]}
```

Error handling:

Query error or other server error: throw InternalServerErrorException with

```
{ "statusCode": 500,
  "message": "A system error has occurred, please try again later."}
```

If the access_token is invalid or expired, JwtAuthGuard will automatically throw an UnauthorizedException (HTTP 401).

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build an account list interface based on Figma's design.

Create the AccountListPage component using React TypeScript and Tailwind CSS. The interface displays a list of cards, each card showing the following information: Bank name, Account type, Balance, and the last four digits of the account number.

Strict design based on Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5320&t=cxgXQfkGoLMfxW48-4> , layout, colors, and fonts match 100%.

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

In AccountListPage, add state:accounts to save the list of accounts, isLoading for the loading status, and error to save the error message.

Implement the asynchronous fetchAccounts function:

Send request: GET /api/v1/accounts with header: Authorization: Bearer <access_token>.

If successful, update the state: accounts = response.accounts, isLoading = false, Delete the old error message

If the accounts array is empty, display a message to the user.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling, loading status, and FE validation.

Loading Status: When isLoading = true, display Spinner or Skeleton Loader instead of the account list.

API Error:

401 Unauthorized: redirects the user to the Login page.

Server error (500): displays the message: "A system error has occurred, please try again later."

Successful but empty list: displays the message *"You haven't linked any accounts. Please add an account,"* along with a button/link to the add account page.

3. Validation FE: Not applicable, as this is a data viewing use case, not a user input form.

Add a bank account

Use case specification

Table 6 presents the Add Bank Account use case specification, which allows a user to add a new account to the system.

Table 6. Use Case Specification: Add New Bank Account

Use case name	Add a new bank account
Description	• Allows logged-in users to link and add information about a new bank account or e-wallet to the system for financial tracking and management.

Actor	User
Main flow	Users access the "Account" page and click "Add account". The user enters the new account information: Bank/Wallet Name, Account Type, Account Number, Branch (optional), Initial Balance. Press the "Add account" or "Save" button. The front-end sends data to the backend API. The backend verifies the access_token and extracts the user_id. The backend validates the input data. If valid, the backend saves the new account information to the Accounts table, associated with user_id. The backend returns the details of the newly created account. The front-end displays a success message and redirects the user to the View Account List page.
Alternative flow	A.1. Invalid input data: If any data field violates the verification rule (e.g., negative balance, blank bank name): display a detailed error message next to the faulty field and keep the user on the Add Account screen. A.2. Duplicate Account Number Error (optional): If the account number already exists with the user: display the message "This account already exists in your list." A.3. Backend system errors: If an error occurs during the data saving process: display the message "Unable to add account at this time. Please try again later."
Preconditions	Users must log in. The user is currently accessing the "Account" page.

Postconditions	Success: The new account is saved to the system, and the user is redirected to the account list page or confirmation page. Failure: An error message about invalid data or a system error is displayed, and the user remains on the Add Account screen.
----------------	---

UI design in Figma

The design is compatible with the Bank Account page interface, with a new add form including: Bank_name, Account_type, Branch_name, Full account number, and Initial balance.

API Contract

Endpoint:

POST /api/v1/accounts

Sample Request:

```
{"bank_name": "TPBank", "account_type": "Checking", "branch_name": "District 4", "account_number_full": "9704221122334455667", "balance": 2500000}
```

Response successful:

```
{"message": "Account created successfully", "account": { "id": 9, "user_id": 1, "bank_name": "TPBank", "account_type": "Checking", "branch_name": "District 4", "account_number_last_4": "5667", "balance": 2500000}}
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling for the account adding functionality.

Create a POST endpoint /api/v1/accounts in NestJS AccountsController. This endpoint must be protected by *JwtAuthGuard* .

Processing logic in AccountsService:

Get the DTO containing account data and user_id from request.user (JwtAuthGuard).

Use ValidationPipe to validate DTOs:

bank_name cannot be empty, account_number_full cannot be empty, balance must be a number and ≥ 0

Check if account_number_full already exists for this user.

If valid and not a duplicate, save the new account information to the Accounts table associated with user_id.

Returns the newly created account object.

Error handling:

Invalid data: ValidationPipe automatically throws BadRequestException (400).

Duplicate account number: throw ConflictException (409) with

```
{ "statusCode": 409,
```

```
"message": "This account already exists in your list."
```

Error saving to database: throw InternalServerErrorException(500) with

```
{ "statusCode": 500,
```

```
"message": "Unable to add account at this time."
```

Please try again later.

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build an account addition interface that accurately matches the design.

Create the AddAccountForm component using React TypeScript and Tailwind CSS. This component should display a form with the following fields: Bank (bank_name), Account Type (account_type), Branch (branch_name, optional), Full Account Number, Initial Balance (balance, number), and an *Add Account button*.

Strict design according to Figma MCP: layout, font, and colors match the design 100%.

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

In the AddAccountForm component:

Create state for the form fields: bank_name, account_type, branch_name, account_number_full, balance.

Implement the asynchronous handleSubmit function to send a request to POST /api/v1/accounts with a payload from the state.

Actions to take after success:

Display the toast message: "Account added successfully".

Redirect the user to the account list page (/accounts).

(Optional) Update the global state to add new accounts without reloading the page.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling, Loading status, and FE validation.

Loading status: Add the state `isLoading`. When submitting, set `isLoading = true` and disable the `Add Account` button. After the API is complete, set `isLoading = false` again.

API Error:

409 Conflict: "This account already exists in your list."

500 Internal Server Error: "Unable to add account at this time. Please try again later."

400 Validation Error: Displays a detailed message below the faulty input field.

FE Validation: Before calling the API:

The 'Bank' and 'Account Number' fields cannot be left blank.

'Initial balance' must be a number and ≥ 0 .

Edit account

Use case specification

Table 7 describes the Edit Account use case, which allows users to update information for an existing account.

Table 7. Use Case Specification: Edit Account Information

Use case name	Edit account information
Description	Allows users to update information fields of an existing bank account or e-wallet, including bank name, account type, branch, or balance, in order to maintain data accuracy within the system.
Actor	User

Main flow	Users access the "Bank Account" page. Users select the account they want to edit and click "Details". On the "Account Details" page, click the "Edit" button. The system displays the edit form with the current data. The user modifies the necessary fields (branch_name, account_type, balance, etc.). Press the "Save changes" button. The front-end calls the backend API: PUT /v1/accounts/:id with the modified data. The backend verifies the access_token and extracts the user_id. The backend checks account ownership. The backend checks the validity of the new data (balance $\geq$ 0, valid account number). If valid, the backend updates the information in the Accounts table. • The backend returns the information that the account has been successfully updated (HTTP 200 OK). The front-end displays a "Update successful" message and updates the account details interface.
-----------	--

Alternative flow	A.1. Invalid input data: The backend returns a Bad Request (HTTP 400) error with details. The front-end displays an error message below the faulty field. The user remains on the editing screen to make corrections. A.2. Authorization error (no editing permission): If the account is not owned by user_id: The backend returns a Forbidden (HTTP 403) error. The front-end displays the message "You do not have permission to edit this account information." A.3. Backend system error: If an error occurs while updating the database: display the message "An error occurred while saving data. Please try again later."
Preconditions	Users must log in. The account to be edited must exist and be owned by the user. The user is currently on the "Account Details" page for that account.
Postconditions	Success: Account information has been updated, and a success message has been displayed. Failure: An error message about invalid data or a system error is displayed, and the user remains on the editing screen.

UI design in Figma

Suitable for the Account Details page interface, the form design includes fields for filling in information to be edited: Bank, Account Type, Branch, Full Account Number, Current Balance (VND), [Cancel] [Save Changes].

API Contract

Endpoint:

PUT /api/v1/accounts/:id

Sample request: {"bank_name": "Vietcombank", "account_type": "Checking", "branch_name": "District 3", "account_number_full": "9704221234567890123", "account_number_last_4": "0123",

```
"balance": 4500000}
```

Response successful:

```
{"message": "Account updated successfully", "account": { "account_id": 1, "user_id": 1, "bank_name":  
"Vietcombank", "account_type": "Checking", "branch_name": "District 3", "account_number_full":  
"9704221234567890123", "account_number_last_4": "0123", "Balance": 4,500,000, "updated_at":  
"2025-11-01T15:25:00.000Z"}}
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server-side error handling for account editing functionality.

Create endpoint `PUT /api/v1/accounts/:id` in `AccountsController` NestJS.

This endpoint must be protected by `JwtAuthGuard`.

Processing logic in `AccountsService`:

Extract `user_id` from `request.user` (`JwtAuthGuard`).

Search for the bank account by `:id`. If not found, throw a `NotFoundException`.

Verify ownership: check `account.user_id === user_id`.

Data Input Validation (DTO): Required fields must not be empty, `balance ≥ 0`.

Update account information in the database.

Returns the updated account object.

Error handling:

Invalid data: `BadRequestException` (400) with

```
{ "statusCode": 400,  
"message": ["balance must not be less than 0"],  
"error": "Bad Request" }
```

User does not own the account: `ForbiddenException` (403) with

```
{ "statusCode": 403,  
"message": "You do not have permission to edit this account information."  
"error": "Forbidden" }
```

Database update error: `InternalServerErrorException` (500) with

```
{ "statusCode": 500,  
"message": "An error occurred while saving the data. Please try again later." }
```

"error": "Internal Server Error" }

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build an account editing interface that accurately matches the design.

Create the AccountEditForm component using React TypeScript and Tailwind CSS. This component should display:

Title: Edit [Bank Name] Account

The form includes the following fields: Bank, Account Type, Branch, Full Account Number, Current Balance.

Two buttons: Cancel and Save Changes.

Strict design according to Figma MCP, ensuring 100% accuracy in layout, spacing, and color.

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

In the AccountEditForm component:

Create state to manage form data: formData, isLoading, error.

Implement the asynchronous handleSaveChanges function to send a PUT request `/api/v1/accounts/:id` with a payload from the state.

Actions to take after success:

Display the toast message: "Update successful".

Update the account details state with the newly received data.

Close the edit form/modal.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling, Loading status, and FE validation.

Loading status: When `isLoading = true`, disable the 'Save changes' button and display the spinner.

API Error:

400 (Bad Request): Displays an API error message directly below the input field.

403 (Forbidden): toast "You do not have permission to edit this account information."

500 (Server Error): toast "An error occurred while saving data. Please try again later."

Validation FE:

The 'Bank' and 'Account Type' fields cannot be left blank.

'Current balance' must be a number and ≥ 0 .

Delete bank account

Use case specification

Table 8 details the Delete bank account use case, allowing users to remove an account they no longer use.

Table 8. Use Case Specification for Deleting Account Information

Use case name	Delete the account information.
Description	Allows users to completely remove an unused bank account or e-wallet. This action will permanently delete the account and all transactions recorded through it.
Actor	User
Main flow	Users access the "My Account" page (list of all accounts). Users can click the "Delete" button on a specific account or go to "Account Details" and click "Delete". The system displays a confirmation dialog warning about the permanent deletion of the account and all related transactions. The user confirms they want to delete it. Front-end calls Backend API: DELETE api/v1/accounts/:id. The backend verifies the access_token and extracts the user_id. The backend checks account ownership. If valid, the backend deletes all related transactions from the Transactions table and removes the account from the Accounts table. The backend returns a success response (HTTP 200 OK). The front-end updates the list of displayed accounts (accounts that have just been deleted will disappear).

Alternative flow	A.1. The user cancels the operation: After the confirmation dialog, if the user clicks "Cancel" or closes the dialog, the deletion operation is canceled and the user remains on the account list page. A.2. Authorization error (no delete permission): If the account does not belong to user_id: The backend returns a Forbidden (HTTP 403) error, and the frontend displays the error message "You do not have permission to perform this operation." A.3. Backend System Errors: If a database error or processing error occurs during deletion, the system rolls back any changes and displays the message "A system error occurred; the account and related transactions could not be deleted."
Preconditions	Users must log in. The user is currently on the "My Account" page. The account to be deleted must exist and be owned by the user.
Postconditions	Success: The account and all related transactions are deleted, and the front-end updates the account list. Failure: An error message is displayed; the account and transaction remain unchanged.

UI design in Figma

Matches the interface of the My Account page.

API Contract

Endpoint:

DELETE /api/v1/accounts/:id

Response successful:

```
{"message": "Account deleted successfully", "deleted_account_id": 1}
```

Response on error: {"statusCode": 404, "message": "Account not found or not owned by current user"}

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build an API service, business logic, and server error handling for the account deletion function.

Create endpoint DELETE /api/v1/accounts/:id in AccountsController NestJS.

This endpoint must be protected by *JwtAuthGuard* .

Processing logic in AccountsService:

Get accountId from path parameter and userId from request.user (JwtAuthGuard).

Find an account in the database by accountId.

Check ownership: if the account exists but account.userId !== userId, throw an error.

Perform a deletion within a transaction:

Delete all related records in the Transactions table with accountId.

Next, delete the account record in the Accounts table.

Returns a successful result.

Error handling:

The account does not exist or is not owned by the user: NotFoundException (404) with JSON:

```
{"statusCode": 404,
```

```
"message": "Account not found or not owned by current user"}
```

Database error when deleting: InternalServerErrorException (500) with message:

```
"A system error occurred; it was not possible to delete the account and related transactions."
```

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build an accurate account deletion confirmation interface based on Figma MCP.

Create a DeleteAccountModal component using React TypeScript and Tailwind CSS with the following elements:

Title: "Account Deletion Confirmation"

Warning icon

Animated warning message: "WARNING: Are you sure you want to delete account [Bank Name] - [**** Last Digit]? This action will PERMANENTLY delete the account and ALL related transactions."

Two action buttons:

"Cancel" to close the modal

"Confirm Delete" (red) performs the action.

The design adheres strictly to Figma MCP standards, ensuring 100% accuracy in layout, font, color, and spacing.

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

In the DeleteAccountModal component: Create management states: isLoading, error. Implement the asynchronous function handleDeleteAccount(accountId) to send a request DELETE /api/v1/accounts/:id.

Actions to take after success:

Close the DeleteAccountModal modal.

Display toast/notification: "Account successfully deleted."

Update the global state or call the `fetch account list` function again to remove the deleted account from the interface.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling) Objective: Complete error handling, Loading status, and FE Validation.

Loading status:

Before calling the API, set isLoading = true.

After the API is complete, set isLoading = false.

When isLoading = true, disable both buttons and display the spinner on the "Confirm Delete" button.

API Errors: Catch errors using try...catch and display the exact error message returned from the backend (e.g., 403, 404, 500) directly inside the modal, above the buttons.

Validation FE: No client input validation is required in this use case.

View bank account details

Use case specification

Table 9 describes the use case "View bank account details," which displays detailed information and the most recent transactions of the account.

Table 9. Table 1.4: Use Case Specification View account details

Use case name	View bank account details
Description	Allows users to view all the details of a specific bank account or e-wallet, including basic account information and a list of the most recent transactions.

Actor	User
Main flow	Users access the "Bank Accounts" page (list of accounts). Users select an account and press the "Details" button. Front-end calls backend API. The backend verifies the access_token and extracts the user_id. The backend checks account ownership. The backend queries account details and the 5 most recent transactions. The backend returns account details and a list of transactions. The front-end displays the account details page on the interface.
Alternative flow	A.1. Account not found (invalid ID): The backend returns a Not Found (HTTP 404) error. The front-end displays the message "This account was not found" and redirects the user to the account list page. A.2. Accounts with no transactions: If the transaction list is empty, display the normal details and the message "No transactions have been recorded for this account." A.3. Authorization error (no viewing permission): If the account is not owned by user_id: The backend returns a Forbidden (HTTP 403) error. The front-end displays the message "You do not have permission to view this account information."
Preconditions	Users must log in. The account whose details need to be viewed must exist and be owned by the current user.

Postconditions	Success: Displays the account details page, including basic information and a list of recent transactions. Failure: An error message is displayed, the user remains on the previous page, or is redirected to the account list.
----------------	---

UI design in Figma

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=416-7878&t=cxgXQfkGoLMfxW48-4> ---Financial-Management-

API Contract

Endpoint:

GET /api/v1/accounts/:id

Request

Header: Authorization: Bearer <access_token>

Successful response:

```
{
  "id": 1,
  "bank_name": "Vietcombank",
  "account_type": "Checking",
  "branch_name": "District 1",
  "account_number_full": "9704221234567890123",
  "Balance": 5200000,
  "recent_transactions": [
    {
      "date": "2025-10-31",
      "amount": -500000,
      "description": "Transfer from Momo",
      "status": "Complete",
      "receipt_id": "RCP001",
      "type": "Expense"
    },
    {
      "date": "2025-10-30",
      "amount": 1,000,000,
      "description": "Transfer from Momo",
      "status": "Complete",
      "receipt_id": "RCP002",
      "type": "Expense"
    }
  ]
}
```

Error handling:

Account not found: NotFoundException (404)

```
{
  "statusCode": 404,
  "message": "This account was not found.",
  "error": "Not Found"
}
```

The account is not owned by the user: ForbiddenException (403)

```
{
  "statusCode": 403,
  "message": "You do not have permission to view this account information.",
  "error": "Forbidden"
}
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling.

Processing logic in AccountsService:

Get accountId from path param and userId from req.user (JwtAuthGuard).

Query the database to find the account by accountId.

If the account is not found, throw a NotFoundException.

Check ownership: if account.userId !== userId, throw ForbiddenException.

If verification is successful, query the 5 most recent transactions related to accountId.

Combine account information and transaction list into a single object and return it.

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build the user interface accurately according to Figma's design.

Create the AccountDetailComponent using React TypeScript and Tailwind CSS, including:

The card section displays detailed account information: bank name, balance, account type, branch, and full account number.

The table or list area displays the 5 most recent transactions with the following columns: Date, Description, Amount, Status.

STRICTLY designed according to Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=416-7878&t=cxgXQfkGoLMfxW48-4>

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

In AccountDetailComponent:

Add state: accountData, isLoading, error.

Implement the asynchronous function fetchAccountDetails(accountId) to send a GET request /api/v1/accounts/:id with the Authorization header.

When the API returns a success message: Call setAccountData(responseData), Set setIsLoading(false), Clear the old error message: setError(null)

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling, loading status, and information display.

Loading state: Before calling the API, setIsLoading(true). When isLoading = true, display a Spinner or Skeleton instead of content.

API Error: Wrap the API in a try...catch block. Display the error message based on the status code.

404: "This account was not found."

403: "You do not have permission to view this account information."

Other errors: "An error occurred, data could not be loaded."

FE Validation: No input validation required. If the transaction array is empty, display:

"No transactions have been recorded for this account."

View Monthly Expenses

Use case specification

Table 10 presents the use case for Viewing Monthly Expenses, providing an overview chart of spending levels by month.

Table 10. Use Case Specification Viewing Monthly Expenses

Use case name	View monthly expenses
Description	• Allows users to visualize and compare total spending for the current month with previous months of the year through charts, helping them understand financial trends and habits.
Actor	User
Main flow	On the Home page, users access the "Spending" page through the navigation bar. The front-end calls the backend API to retrieve total monthly spending data. The backend verifies the access_token and extracts the user_id. The backend queries the Transactions table to retrieve the total spending amount (SUM(Amount)) for each month with type = 'Expense' and the current user_id within a 12-month range. The backend returns total spending data in the format Month - Total Amount. The front-end receives the data and draws a bar chart comparing expenses. The X-axis displays the months of the year, and the Y-axis displays the amount of money spent. Chart column: current month in green, previous months in gray. The interface displays complete charts.

Alternative flow	A.1. No spending data available: • If the query does not return data: display the message "No spending data has been recorded for analysis." A.2. Data is only available for the current month: The chart only displays the green column for the current month, and may display a message encouraging transaction recording. A.3. System error / access error: If an error occurs during querying or validation: The backend returns an error (HTTP 500), the front-end displays a generic error message, and the graph cannot be plotted.
Preconditions	Users must log in. The user has recorded spending transactions (type = 'Expense') for the current month and previous months.
Postconditions	Success: Displayed a monthly spending comparison chart on the "Spending" page. Failure: Displays an error message or a message indicating no spending data.

UI design in Figma

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=66-5698&t=cxgXQfkGoLMfxW48-4> ---Financial-Management-

API Contract

Endpoint:

GET /api/v1/expenses/summary?userId={user_id}

Header: Authorization: Bearer <access_token>

Successful response:

```
{
  "data": [
    { "month": "January", "totalExpense": 1250.00 },
    { "month": "Feb", "totalExpense": 2100.50 },
    { "month": "Mar", "totalExpense": 1900.00 }
  ]
}
```

Error handling:

```
{ "error": "Cannot load data for Expense." }
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling.

Processing logic in ExpensesService:

Get user_id from the JWT payload (@Request()).

Query the Transactions table in the database.

Filter transactions by: type = 'Expense', user_id = current user, Transaction year = current year

Group by month and sum (SUM(amount)).

Convert the result into an array of objects: { month: string, totalExpense: number } with the abbreviated month name.

Returns JSON with the key data.

Error handling: If the DB query fails, throw an InternalServerErrorException with:

```
{ "error": "Cannot load data for Expense" }
```

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: Build a user interface based on Figma's design.

Create the ExpenseSummaryChart component (React TypeScript + Tailwind CSS) which includes: Chart area title. X-axis: Month of the year. Y-axis: Total expenditure. Chart columns with a tooltip that displays the amount when hovered over.

STRICTLY according to Figma MCP design: <https://www.figma.com/design/iEOmfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-K>

its--Community-?node-id=66-5698&t=cxgXQfkGoLMfxW48-4

Column colors: Current month is green. Previous months are gray.

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and process the data.

Add state: summaryData: chart data, isLoading: loading status, error: error message

The asynchronous fetchExpenseSummary function inside useEffect:

Call API GET /api/v1/expenses/summary with Authorization: Bearer

<access_token>

If successful, update summaryData with response.data

The chart is rendered based on summaryData; the current month column is green, and other months are gray.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling and loading status.

Loading:

Before calling the API, set isLoading = true.

In the finally block, set isLoading = false

When isLoading = true, display ChartSkeletonLoader or Spinner.

API Error:

Wrap requests in try...catch

If error (500) occurs, set error = error.response.data.error and display it in the chart area.

In the case of an empty array:

If data.length === 0, display: "No spending data has been recorded for analysis."

If there is only one element (the current month), the chart will still be displayed with a single green column.

Validation FE: Not applicable as there is no input form.

View Expenditure Details by Category

Use case specification

Table 11 details the use case "View Expenses by Category," grouping expenses by category for user convenience.

Table 11. Use Case Specification View expenditure details by category

Use case name	View detailed spending by category.
Description	• Allows users to analyze total spending over a specific period (usually the current month) by breaking down the amount spent into established categories (Housing, Food, Entertainment, etc.).
Actor	User

Main flow	Users access the "Spending" page from the navigation bar. The front-end calls the backend API to retrieve group spending details by category. The backend verifies the access_token and extracts the user_id. The backend queries the Transactions table (type = 'Expense') and combines it with the Categories table to group total spending by category_id for the current and previous months. The backend returns a list of categories, current month's total spending, previous month's total spending, and individual transaction items for each category. The front-end displays a detailed list of expenses by category: Category name and icon, total amount spent in the current month, and percentage increase/decrease compared to the previous month. • Under each category, display a list of smaller transaction items belonging to that category during the current month.
Alternative flow	A.1. No expenses in the current month: If the current month's total spending is zero, display the message: "You have no expenses recorded this month." A.2. System errors / access errors: If an error occurs during querying or authentication: The backend returns an error (HTTP 500), the front-end displays a generic error message, and data cannot be parsed.
Preconditions	Users must log in. The user has recorded spending transactions (type = 'Expense') and these transactions have been assigned a valid category_id. The system has spending data for at least the current month to calculate the comparison rate.

Postconditions	Success: Displayed the Expense Details page with a list of categories, total spending for each category, and a comparison to the previous month. Failure: Displays an error message or a message indicating no spending data.
----------------	---

UI design in Figma

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5698&t=cxgXQfkGoLMfxW48-4>

API Contract

Endpoint:

GET /api/v1/expenses/breakdown?month=YYYY-MM

Sample request:

Header: Authorization: Bearer <access_token> Query Parameters: month - format 'YYYY-MM', e.g. '2025-11'

Successful response:

```
{ "data": [ { "category": "Housing", Total: 250.00, "changePercent": 15, "subCategories": [{
  "item_description": "House Rent", "amount": 230.00, "date": "2025-05-17" }, { "item_description":
  "Parking", "amount": 20.00, "date": "2025-05-17" } ] }, { "category": "Food", Total: 350.00,
  "changePercent": -8, "subCategories": [{ "item_description": "Grocery", "amount": 230.00, "date":
  "2025-05-17" }, { "item_description": "Restaurant Bill", "amount": 120.00, "date": "2025-05-17" } ] } ] }
```

Error handling:

```
{ "error": "No input data for this month." }
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling.

Processing logic in ExpensesService:

Get userId from the JWT payload (@Request()) and month from the query parameters.

The previous month is calculated based on the current month.

Execute two queries in parallel on the Transactions table:

Query transactions of type='Expense' in the current month.

Query the transactions of type='Expense' in previousMonth.

Group the results by category_id and calculate the total for each category.

Loop through the current month's categories:

Calculate $\text{changePercent} = \frac{\text{currentTotal} - \text{previousTotal}}{\text{previousTotal}} \times 100$. If previousTotal = 0, it could be 100 or null.

Get a list of subcategories within a category.

If there are no transactions in the current month, throw a NotFoundException.

Returns JSON with the key data.

Error handling: If no transactions are found, throw a NotFoundException with:

```
{ "error": "No input data for this month." }
```

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: Build a user interface based on Figma's design.

Create ExpensesBreakdown component using React TypeScript + Tailwind CSS.

Each card displays:

Icon and category name

Total expenditure of the portfolio

The percentage change (changePercent) is shown in green if it increases and red if it decreases.

A list of sub-transactions, each line containing: item_description, date, and amount.

STRICTLY according to Figma MCP design:
<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5698&t=cxgXQfkGoLMfxW48-4>

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

- Add states:

expensesData: breakdown data

isLoading: loading status

error: error message

- Implement the asynchronous function fetchExpensesBreakdown(month: string):

API call GET /api/v1/expenses/breakdown?month=YYYY-MM with the Authorization header.

If successful, set expensesData = response.data.

Set isLoading = false and clear the old error (error = null).

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling and loading status.

Loading: Before calling the API, set `isLoading = true`. While loading, display the Skeleton Loader like a real layout card.

API Error: Wrap the request in a `try...catch`. If a 404 error occurs, set `error = "No spending data for this month."` and display a message in the interface. If no data is found, the message should be: "You have no expenses recorded this month."

Validation FE: Only call `fetchExpensesBreakdown` with a valid month string (YYYYMM) from Date Picker.

View Upcoming Invoices List

Use case specification

Table 12 describes the use case "View Invoice List," which displays recurring invoices that are due soon for the user to manage.

Table 12. Use Case Specification View upcoming invoice list

Use case name	View the list of upcoming bills.
Description	• Allows users to track and manage upcoming recurring bills, helping them proactively prepare their finances and avoid late payments.
Actor	User

Main flow	Users access the Bills page through the navigation menu. The front-end calls the backend API to retrieve a list of upcoming invoices (GET /api/v1/bills). The backend verifies the access_token and extracts the user_id. The backend queries a list of recurring invoices belonging to user_id with due dates in the near future. The list of invoices is sorted by due date in ascending order (due date ASC). The backend returns a list of invoices, including: Package name, Provider logo, Description, Due date, Final payment date, and Amount. The front-end receives the data and displays a list of invoices with full details and corresponding logos.
Alternative flow	A.1. No bills due soon: • If the returned invoice list is empty: display a message. "You don't have any bills due soon." Provide a link for the user to set up a new invoice. A.2. System error / access error: If authentication or data query errors occur: The backend returns an error (HTTP 401, HTTP 500), the frontend displays a general error message and does not show the invoice list.
Preconditions	Users must log in. The user has set up at least one recurring invoice that is due soon.
Postconditions	Success: Displaying a list of upcoming invoices on the Invoices page. Failure: Displays an error message or a message indicating no upcoming invoice.

UI design in Figma

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=66-5609> ---Financial-Management-

API Contract

Endpoint:

GET /api/v1/bills

Header: Authorization: Bearer <access_token>

Request Body: None (GET request)

Response successful:

```
{"data": [{"billId": 1, "userId": 1, "itemDescription": "Figma - Yearly Plan", "logoUrl": "https://cdn.figma.com/logo.png", "dueDate": "2025-05-15", "lastChargeDate": "2024-05-14", "Amount": 150.00}, {"billId": 2, "userId": 1, "itemDescription": "Adobe Creative Cloud", "logoUrl": "https://cdn.adobe.com/logo.png", "dueDate": "2025-06-10", "lastChargeDate": "2024-06-10", "Amount": 559.00}]}
```

Error handling:

```
{"message": "Failed to fetch bills"}
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling.

Processing logic in BillsService:

Get user_id from the JWT payload (@Request()).

Query the database to retrieve all invoices belonging to user_id.

Filter invoices with a due date $\geq$ the current date.

Sort the results by dueDate ASC.

Returns JSON with the key data.

Error handling: If an error occurs during the database query, throw an InternalServerErrorException with:

```
{"message": "Failed to fetch bills"}
```

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build the user interface accurately according to Figma's design.

Component: UpcomingBills (React TypeScript + Tailwind CSS)

Display a list of upcoming invoices:

Supplier logo, Invoice description (itemDescription), Due date, Amount, Pay Now button

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

Add the following states: bills: list of invoices, isLoading: loading status, error: error message

Asynchronous function fetchUpcomingBills:

Call the GET API /api/v1/bills with the Authorization header.

If successful:

set bills = response.data

If data.length === 0, display the message: "You do not have an invoice."

Which one is due for payment soon?

set isLoading = false and error = null.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling and loading status.

Loading: Before calling the API, set isLoading = true. When the request ends, set isLoading = false.

Display a spinner or skeleton loader during the loading process.

API Error: If the API returns an error (e.g., 500), update the error state with the following message:

"Unable to load invoice list. Please try again." This message will be displayed at the invoice list location.

Validation FE: Not required, as this is a read-only function.

View the list of Goals

Use case specification

Table 13 presents the use case for Viewing Monthly Goals, which displays financial goals and their completion status.

Table 13. Use Case Specification See List of Goals

Use case name	View the list of goals.
Description	Allows logged-in users to view an overview of all financial goals they have set in the system, including overall savings goals and spending limits by category.

Actor	User
Main flow	The user is logged in and has a valid access token. Users access the Goals page from the navigation bar (/goals). Front-end calls Backend API: GET /api/v1/goals. The backend verifies the access_token and extracts the user_id. The backend queries the Goals and Transactions tables based on user_id, calculating the Target Achieved = (Total Income - Total Expenses) for the current month. The backend returns a list of the user's goals (including savings goals and spending limits) for the current month. The front-end receives data and displays a detailed list of objectives on the user interface.
Alternative flow	A.1. No goals have been set: • If the target list is returned empty, display the following message: "You haven't set any goals yet. Let's create your first goal!" Provide a button or link for users to start creating new goals. A.2. Authentication/Access Errors: If the access_token is invalid or has expired: The backend returns an Unauthorized (HTTP 401) error, and the frontend redirects the user to the login page. A.3. Backend system errors: If a database query error or internal processing error occurs, display a friendly error message: "A system error occurred while loading the target, please try again later."
Preconditions	• Users must be logged in (with a valid access token).

Postconditions	Success: Display a list of the goals you have set on the Goals page. Failure: Displays an error message or a message indicating that no target has been set.
----------------	--

UI design in Figma

[https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=66-5 829&t=cxgXQfkGoLMfxW48-4](https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=66-5%20829&t=cxgXQfkGoLMfxW48-4) ---Financial-Management-Dashboard-UI-Kits--Community-?node-id=66-5 829&t=cxgXQfkGoLMfxW48-4

The Savings Goal section displays: Target Achieved, Target Amount, and an “Adjust Goal” button. The Expenses Goals by Category section displays category tags (Housing, Food, Transportation, etc.).

API Contract

Endpoint:

GET /api/v1/goals

Request:

Header: Authorization: Bearer <access_token>

Request Body: No successful response received:

```
{ "savingGoal": { "goal_id": 1, "goal_type": "Saving", "target_amount": 60000000, "target_achieved": 10000000, "start_date": "2025-01-01", "end_date": "2025-12-31"}, "expenseGoals": [ {"goal_id": 2, "category": "Food", "target_amount": 3000000}, {"goal_id": 3, "category": "Transportation", "target_amount": 2000000}] }
```

Error handling:

```
{"statusCode": 500, "message": "500 Internal Server Error. Try again."}
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling.

The endpoint GETs /api/v1/goals in GoalsController and is protected by JwtAuthGuard.

Get user_id from the JWT payload.

Query the database to retrieve:

Savings Goal

Expense goals for user_id in the current month.

Calculate target_achieved for savingGoal: $\text{target_achieved} = \text{Total Income (current month)} - \text{Total Expenses (current month)}$

Return data in the response format.

If an error occurs during the database query or logic processing, throw an `InternalServerErrorException` with the JSON error.

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build the user interface accurately according to Figma's design.

Component: `GoalsPage` (React TypeScript + Tailwind CSS)

Display:

"Savings Goal" area: Target Achieved, Target Amount, "Adjust Goal" button

The "Expenses Goals by Category" section: a list of spending goals categorized by category.

- STRICTLY according to Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard-UI-Kit>
s--Community-?node-id=66-5829

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

- Add states: `goalsData` – target data, `isLoading` – load status, error
– error message

- Asynchronous `fetchGoals` function:

Call the GET API `/api/v1/goals` with the Authorization header.

If successful:

```
set goalsData = response.data
```

```
set isLoading = false
```

If both `savingGoal` and `expenseGoals` are empty, display the message: "You haven't set any goals yet. Please create your first goal!"

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To improve error handling and loading status.

Loading: Before calling the API, set `isLoading = true`. While loading, display Spinner or Skeleton Loader.

API Error:

If you receive a 401 Unauthorized error, navigate to the Login page.

If a system error occurs (e.g., 500), update the error and display: "A system error occurred while loading the target, please try again later."

Validation FE: Not applicable, as this is a data viewing function.

Create a New Goal

Use case specification

Table 14 details the Create New Goal use case, which allows users to set a new financial goal.

Table 14. Use Case Specification Create New Goal

Use case name	Set new goals
Description	Allows users to set a new financial goal, such as an overall savings goal or a category-specific spending limit, by entering parameters such as goal type, amount, and timeframe.
Actor	User
Main flow	Users access the Goals page from the navigation bar and click the "Add new goal" button. The system displays an input form with the following information: Goal_type, Category (if needed), Target_amount, Start_date, and End_date. The user enters the information and clicks the "Save" button. The front-end sends requests with data to the back-end API. The backend verifies the access_token and extracts the user_id. The backend checks the validity of the input data: amount > 0, start_date < end_date, and category validity if required. If valid, the backend saves the new goal to the Goals table, linked to user_id. The backend returns a success response. The front-end receives feedback, displays a "Goal created successfully" message, and updates the list of goals.

Alternative flow	A.1. Invalid input data (Validation Error): If any data field violates the rule, the backend returns a Bad Request error (HTTP 400). The front-end displays the corresponding error message below the faulty field, and the user edits it. A.2. Spending objectives without selecting a category: If goal_type is Expense Limit but the Category field is empty, display the error message: "Please select an expense category."
Preconditions	• Users must log in. • Categories must be available if you want to create spending goals.
Postconditions	Success: The new goal has been saved to the Goals table, and the system has updated the list of goals. Failure: Displays an invalid data error message, user remains on the Create New Goal form.

UI design in Figma

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=416-6052>

---Financial-Management-

API Contract

Endpoint:

POST /api/v1/goals

Request:

Header: Authorization: Bearer <access_token>

Request Body sample:

```
{ "goal_type": "Saving", "category_id": null, "start_date": "2025-05-01", "end_date": "2025-12-31", "target_amount": 80000000 }
```

Response successful (HTTP 201):

```
{ "message": "Goal created successfully", "goal_id": 9 }
```

Error handling:

```
{"statusCode": 400, "message": "End date must be < start date."}
```

```
{"statusCode": 500, "message": "Cannot add Goals right now. Try again."}
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling.

The POST endpoint `/api/v1/goals` is located in `GoalsController` and protected by `JwtAuthGuard`.

Get the DTO containing the target data; `user_id` is obtained from the `access_token`.

Validation: `target_amount > 0`, `end_date` after `start_date`, if `goal_type = "Expense"`, `category_id` must not be null and must be valid.

If valid, create a new record in the `Goals` table associated with `user_id` and save it to the database.

Error handling:

Validation failed: `BadRequestException` (HTTP 400)

System error: `InternalServerErrorException` (HTTP 500)

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build the user interface accurately according to Figma's design.

Component: `CreateGoalModal` (React TypeScript + Tailwind CSS)

Forms in modals:

Title: "Add a new goal"

Select Goal Type (Radio/Dropdown: "Overall Savings", "Expense Limit")

Select a Category (shown if the goal is "Spending Limit")

Enter the target amount (currency input)

Select the Start Date and End Date (Date Picker)

Action buttons: "Save", "Cancel"

• STRICTLY according to Figma MCP:
<https://www.figma.com/design/iEOnfperOrck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=416-6052> ---Financial-Management-

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

States: `goalType`, `categoryId`, `targetAmount`, `startDate`, `endDate`, `isLoading`, `error`

The handleSubmit function:

Send a POST request to /api/v1/goals with the payload retrieved from the state.

If successful:

Display toast: "Setting successful goals."

Close modal

Call the callback to refresh the target list.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: To finalize error handling, loading status, and final validation.

Loading:

The state variable isLoading

When starting the API call: `isLoading = true`, disable the "Save" button, display the spinner.

When API terminates: isLoading = false

API Error:

Wrapped in try...catch

Display the error message from error.response.data.message if available. If not, display: "Could not create target at this time. Please try again later."

FE Validation: Before calling the API:

target_amount must be greater than 0.

start_date and end_date must be selected.

end_date must come after start_date

If the target is "Expense", category_id must be selected.

If validation fails, display an error message under the corresponding field and do not call the API.

Adjusting Monthly Goals

Use case specification

Table 15 describes the use case specification for Adjusting Monthly Goals, supporting the updating of goal values and progress.

Table 15. Use Case Specification Adjusting Monthly Goals

Use case name	Adjust Goal
---------------	-------------

Description	Allows users to update the target amount or the achieved amount of an existing goal (savings or spending) to reflect changes in their financial plan.
Actor	User
Main flow	Users access the Goals page from the navigation bar and select the goal they want to adjust. Users click the "Adjust Goal" button on the corresponding category tab. The system displays a popup form with pre-filled information, including editable fields (e.g., Target Amount). The user changes the values of the necessary fields. The user clicks the "Save" button. The front-end sends new data to the backend API. The backend verifies the access_token and extracts the user_id. The backend checks ownership and validity of the new data (Target Amount > 0). If valid, the backend updates the goal in the Goals table. The backend returns a success response. The front-end displays the message "Target adjustment successful". The popup closes and the target interface is updated with the new value.

Alternative flow	A.1. Invalid input data (Validation Error): If any data field violates the rules (e.g., entering a negative value), the backend returns a Bad Request error (HTTP 400). The front-end displays an error message in a popup, and the user edits it. A.2. Authorization error (No editing rights): If the target is not owned by the current user_id, the backend returns a Forbidden error (HTTP 403). The front-end displays the error message: "You do not have permission to edit this target." A.3. Backend system errors: If an error occurs during the data update process, the system displays the general message: "Could not save changes at this time." Please try again later.
Preconditions	Users must log in. The target that needs adjustment exists and is owned by the user.
Postconditions	• Success: The Target Amount or Archived Amount information is updated in the Goals table, and the Goals interface is updated. • Failure: An error message is displayed, and the user remains in the target adjustment popup.

UI design in Figma

Matches the design of the Objectives page interface.

API Contract

Endpoint:

PUT /api/v1/goals/{goalId}

Request:

Header: Authorization: Bearer <access_token>

Request Body sample:

```
{ "target_amount": 70,000,000 }
```

Response successful (HTTP 200):

```
{"message": "Goal updated successfully", "updated_goal": {"goal_id": 1, "target_amount": 70,000,000}}
```

Error handling:

```
{"statusCode": 400, "message": ["target_amount must be a positive number"], "error": "Bad Request"}
```

```
{"statusCode": 403, "message": "You do not have permission."}
```

```
{"statusCode": 500, "message": "Cannot update now. Try again."}
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling.

The endpoint PUT /api/v1/goals/{goalId} in GoalsController is protected by JwtAuthGuard.

Get goalId from params, user_id from JWT, and DTO containing target_amount.

Search for the target in the database by goalId. If not found, throw a NotFoundException.

Check ownership: goal.userId must match user_id. Otherwise, throw a ForbiddenException.

Validation: target_amount > 0 (use class-validator in DTO).

If valid, update target_amount in the database.

Returns a success response.

Error handling:

DTO validation failed: BadRequestException (HTTP 400)

Not owned by: ForbiddenException (HTTP 403)

DB Error: InternalServerErrorException (HTTP 500)

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build the user interface accurately according to Figma's design.

Component: AdjustGoalModal (React TypeScript + Tailwind CSS)

Modal contains:

Title: "Adjusting Goals"

The input field for "Target Amount" is a numeric field; the current value should be pre-filled.

Primary button: "Save"

Use the secondary button or the 'X' icon to close the modal button.

3. Prompt C: Generate Frontend Logic Code (State & API Connections)

Objective: Connect the UI to the API and successfully handle the workflow.

State: targetAmount, isLoading, error

The handleSave function:

Send a request to PUT /api/v1/goals/{goalId} with the payload: { target_amount: }

targetAmount

If successful:

Displayed toast: "Target adjustment successful."

Call onClose() to close the modal.

Call the onSuccess() callback to refresh the list of targets.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling) Objective: Complete error handling, loading status, and final validation.

Loading:

State variable: isLoading

When starting the API call: `isLoading = true`, disable the "Save" button, display the spinner.

When API terminates: isLoading = false

API Error:

Wrapped in try...catch

Status 400: Displays the error "Target amount must be greater than 0." under input.

Status 403: Displays the error "You do not have permission to edit this target" in the general area.

Other errors (500): display "Cannot save changes at this time. Please try again later."

FE Validation: Before calling the API:

targetAmount must be greater than 0

Otherwise, display an error message under the input and do not call the API.

View Savings Summary Chart

Use case specification

Table 16 describes the use case "View Savings Overview Chart," which displays a chart comparing net savings by month.

Table 16. Use Case Specification See Summary Chart Savings

Use case name	View the Savings Overview chart.
Description	Allows users to visualize their net savings (Total Income - Total Expenses) for each month of a selected year, and compare it with savings results from the same period of the previous year.
Actor	User
Main flow	Users access the Goals page from the navigation bar. Users select the year they want to view data for in the Savings Overview section (if not selected, the default is the current year). The front-end sends requests to the backend API. The backend verifies the <code>access_token</code> and extracts the <code>user_id</code>. Backend computation: Current month savings: For each month of the selected year, calculate (Total Income - Total Expenses) from the table. Transactions. Comparative savings: similarity calculation for the same months of the previous year. The backend groups data by month and returns chart results (including monthly savings for the current year and the previous year). The front-end receives data and displays a line graph. Chart): X-axis: months (1 to 12). Y-axis: Net savings. The chart displays at least two lines: the selected year and the previous year. The interface displays complete charts for the user.

Alternative flow	A.1. No transaction data is available for the selected year: • If there are no Revenue or Expense transactions in the selected year, the front-end displays the message: "There is no transaction data available for this year to calculate savings."
Preconditions	Users must log in. The user recorded Revenue and Expense transactions during the months of the selected year.
Postconditions	Success: The line graph shows the total net savings by month on the Savings Overview page. Failure: Displays an error message or a message indicating there is no data to display.

Figma UI integration:

<https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank-Dashboard-UI-Kits--Community-?node-id=66-5829&t=cxgXQfkGoLMfxW48-4> ---Financial-Management-

Dropdown for selecting a year (e.g., “2024”, “2025”)

Line chart: X-axis: January → December. Y-axis: Total savings. Bold line (blue): “This Year”

Faint (gray) line: “Same period last year”

API Contract

Endpoint:

GET /api/v1/savings/summary

Sample request:

GET /api/v1/savings/summary?year=2025

Query Params: year (optional): The year you want to view the savings overview for. If not specified, the default is the current year.

Sample response:

```
{ "user_id": 1, "year": 2025, "summary": { "this_year": [ { "month": "01", "amount": 1500000 }, { "month": "02", "amount": 2000000 }, { "month": "03", "amount": 2500000 }, { "month": "04", "amount": 2200000 }, { "month": "05", "amount": 2800000 }, { "month": "06", "amount": 3100000 }, { "month": "07", "amount": 3300000 }, { "month": "08", "amount": 2700000 }, { "month": "09", "amount": 2400000 }, { "month": "10", "amount": 2600000 }, { "month": "11", "amount": 3000000 }, { "month": "12", "amount": 3500000 } ], "last_year": [ { "month": "01", "amount": 1200000 }, { "month": "02", "amount": 1800000 }, { "month": "03", "amount": 2000000 }, { "month": "04", "amount": 2200000 }, { "month": "05", "amount": 2400000 }, { "month": "06", "amount": 2600000 }, { "month": "07", "amount": 2800000 }, { "month": "08", "amount": 3000000 }, { "month": "09", "amount": 3200000 }, { "month": "10", "amount": 3400000 }, { "month": "11", "amount": 3600000 }, { "month": "12", "amount": 3800000 } ] }
```

```
"03", "amount": 2000000}, {"month": "04", "amount": 1900000}, {"month": "05", "amount": 2300000}]] }
```

Error handling:

```
{"statusCode": 500,"message": "An internal server error occurred while processing the Savings highlighting."}
```

Coding prompt

1. Prompt A: Generate Backend Code (NestJS Service/Controller)

Objective: To build API services, business logic, and server error handling.

Create a GET endpoint `/api/v1/savings/summary` in `SavingsController`, protected by `JwtAuthGuard`.

Get `user_id` from JWT via `@Request()` req.

Get the year parameter from the query. If it's not included, the default is the current year.

Query the `Transactions` table to retrieve all Revenue and Expense transactions for the months of the year.

Calculate net savings each month: `saving = total_revenue - total_expense`.

If there are no transactions, the value is 0.

Repeat the above steps for the previous year (`year - 1`) and create the `last_year` array.

Return the response in the requested format.

If an error occurs, throw `InternalServerErrorException`.

2. Prompt B: Generate Frontend Code (React Component & UI/MCP)

Objective: To build a user interface that displays the Savings Overview chart according to Figma's design.

Component: `SavingsSummaryChart` (React + TypeScript + Tailwind CSS)

Component layout:

Title: "Saving Summary"

Dropdown menu for selecting years (e.g., 2023, 2024, 2025) – Line chart using the `Recharts` library.

Interface requirements:

The X-axis displays the 12 months (January–December).

Line 1: Current year, dark blue, label "This Year"

Route 2: Last year, light gray, label "Same period last year"

The design must comply with Figma MCP: <https://www.figma.com/design/iE0nfper0rck0R1b0MQBzo/Finebank---Financial-Management-Dashboard>

-UI-Kits--Community-?node-id=66-5829

3. Prompt C: Generate Frontend Logic Code (State & API Connection) Objective: Connect the component to the API and manage data state.

State:

Selected Year: the selected year (default: current year)

chartData: contains this_year and last_year

isLoading: loading status

error: error message

The fetchSavingsSummary(year) function:

Send request GET /api/v1/savings/summary?year=...

If successful, update the chartData state with the data from the API.

Use `useEffect` to call the API whenever `selectedYear` changes.

4. Prompt D: Exception Handling (Client-side Validation & Error Handling)

Objective: Improve UX when calling APIs and when data is missing.

1. Loading:

Before calling the API: isLoading = true

After completion: isLoading = false • During loading: display Spinner instead of chart

2. API Errors:

Catch errors with try...catch

If the API fails, display the message: "Unable to load data. Please try again later."

3. No data available:

- If both this_year and last_year are empty → display: "There is no transaction data for this year to calculate savings."

The Generated Website At Glance

Software Interface

Includes illustrative images of automatically generated screens, corresponding to each use case.



The login screen for FINEbank.IO features a central form with the following elements:

- Header:** The text "FINEbank.IO" is displayed in a large, bold, teal font.
- Email Address:** A text input field with the label "Email Address" above it. The field contains the email address "johndoe@email.com".
- Password:** A password input field with the label "Password" above it. The password is masked with 12 dots. A link labeled "Forgot Password?" is located to the right of the field.
- Remember Me:** A checkbox labeled "Keep me signed in" is positioned below the password field.
- Login Button:** A teal button with the text "Login" is centered below the checkbox.
- Separator:** A horizontal line with the text "or sign in with" in the center.
- Google Sign-In:** A button featuring the Google logo and the text "Continue with Google".
- Create Account:** A link labeled "Create an account" is located at the bottom of the form.

Figure 4. Login screen interface.

Figure 4 illustrates the Account Login screen. The login form includes two required fields: email and password. Upon successful login, the system will automatically redirect to the Home page and display the username.



FINEbank.IO

Full Name

Email Address

Password

Confirm Password

Sign Up

or sign up with

 Continue with Google

Figure 5. Account registration screen interface.

Figure 5 illustrates the Account Registration screen. The registration form includes required fields such as: full name, email, password, and confirm password.

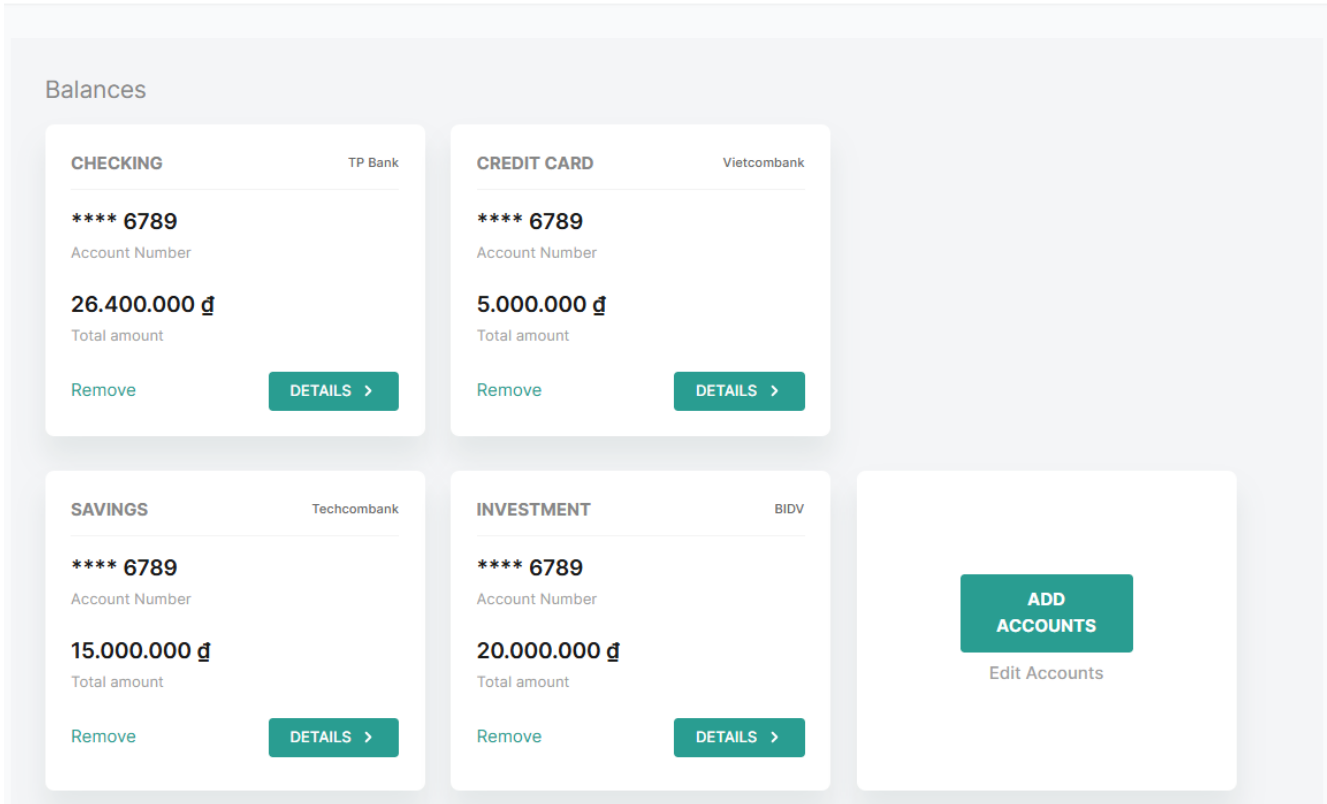


Figure 6. Account list viewing screen interface.

Figure 6 shows the account list view screen. Here, users can view account information, branches, bank names, etc., for all existing accounts, as well as perform actions such as adding, editing, and deleting accounts.

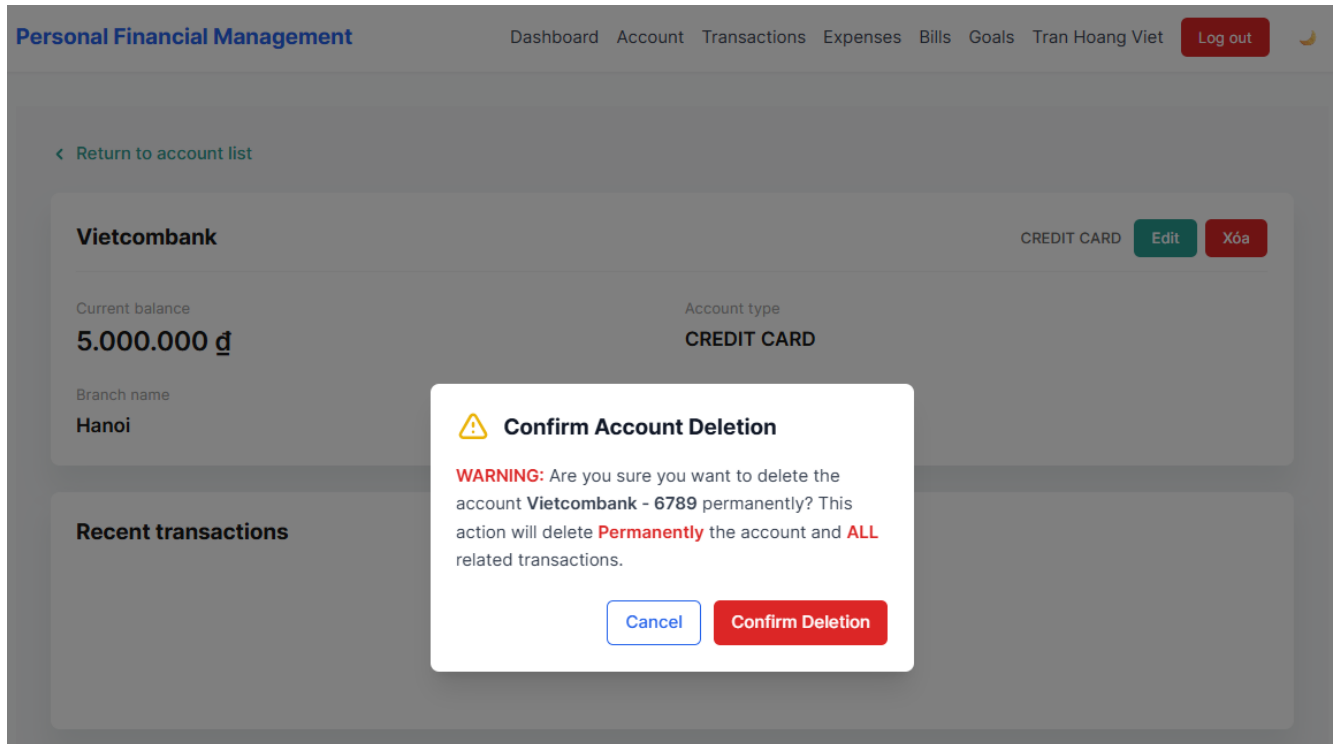


Figure 7. Account deletion screen interface.

Figure 7 illustrates the Account Delete screen. The screen displays a warning; the user clicks "Confirm Delete" if they are sure they want to delete the account.

[< Return to account list](#)

TP Bank

CHECKING

Edit

Xóa

Current balance

26.400.000 đ

Account type

CHECKING

Branch name

Hanoi

Account number

0123456789

Recent transactions

DATE	DESCRIPTION	AMOUNT	STATUS
09/01/2026	Cafe' with friends	-200.000 đ	Complete
08/01/2026	Salary 01/2026	+20.000.000 đ	Complete
07/01/2026	Party with friends	-500.000 đ	Complete
06/01/2026	English class for son	-1.000.000 đ	Complete
05/01/2026	Buy car petrol	-500.000 đ	Complete

Figure 8. Account details viewing screen interface.

Figure 8 illustrates the account details screen. Users can view detailed information for any of their accounts, including: account number, branch name, bank name, account type, current balance, and the 5 most recent transactions for that account.

[< Return to account details](#)

Edit Account TP Bank

Bank

TP Bank

Loại tài khoản

Checking



Branch Name

Hanoi

Full account number

0123456789

Current balance

26400000

Hủy

Save Changes

Figure 9. Account information editing screen interface.

Figure 9 illustrates the account information editing screen. The screen includes editable fields. After making edits, the user clicks Save Changes for the system to update the changes.



Latest Transactions

+ Add transaction

All		Revenue		Expenses	
ITEMS	SHOP NAME	DATE	PAYMENT METHOD	AMOUNT	
Cafe' with friends	Trung Nguyen Legend	09/01/2026	Bank transfer	-200.000 đ	
Salary 01/2026	Add-On Development	08/01/2026	Bank transfer	+20.000.000 đ	
Party with friends	Hem quan	07/01/2026	Bank transfer	-500.000 đ	
English class for son	Apollo English	06/01/2026	Bank transfer	-1.000.000 đ	
Buy car petrol	Petrolimex	05/01/2026	Bank transfer	-500.000 đ	
Power bills	PVN	05/01/2026	Bank transfer	-1.200.000 đ	
Water bill	29 Xuan Phuong Clean Water Plant	05/01/2026	Bank transfer	-200.000 đ	

Figure 10. Transaction list viewing screen interface.

Figure 10 illustrates the Transaction List View screen. The screen displays all transactions sorted by most recent date, with tabs dividing transactions into Income and Expenses.



Add New Transaction

Transaction Name

Example: Movie Ticket

Category *

-- Select category --

Amount

0

Transaction Type

☐ Revenue ☒ Expense

Payment Account *

-- Select account --

Transaction Date

01/07/2026

Shop Name (Optional)

Example: Inox

Payment Method (Optional)

Example: Credit Card

Cancel

Save

Figure 11. Interface for adding a new transaction.

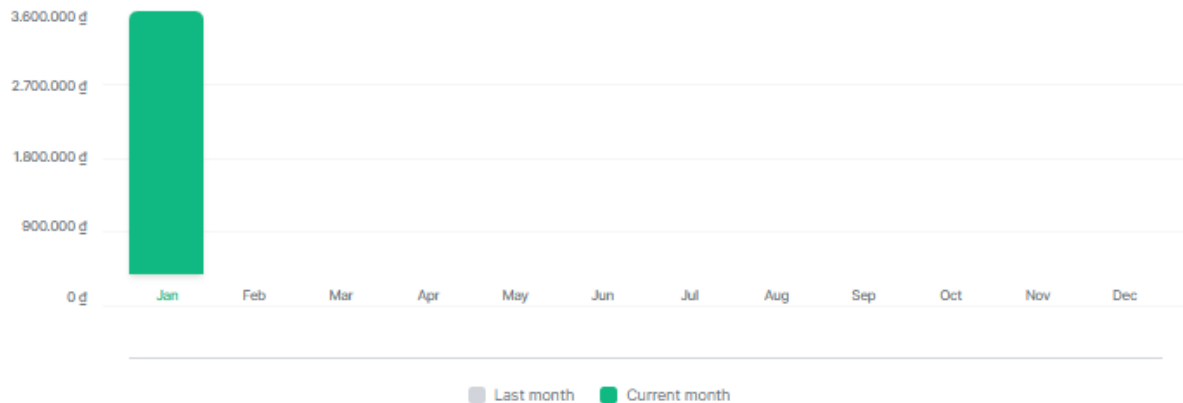
Figure 11 illustrates the Add New Transaction screen. The screen displays a form for the user to enter information such as Transaction Name, Transaction Type, Category, Payment Account, etc. After entering all the necessary information, the user clicks Save to add the new transaction.



Chi tiêu

Monthly Comparison

Compare monthly expenses throughout the year 2026



Expenses Breakdown

Expenditure details by category

Select month: January 2026



Daily Living Expenses

2.400.000 ₫

+100.0%

Buy car petrol	500.000 ₫
05/01/2026	
Power bills	1.200.000 ₫
05/01/2026	
Water bill	200.000 ₫
05/01/2026	
Produce to eat, Ramadan	500.000 ₫

Education & Development

1.000.000 ₫

+100.0%

English class for son 1.000.000 ₫

06/01/2026

Entertainment & Leisure

200.000 ₫

+100.0%

Cafe' with friends 200.000 ₫

09/01/2026

Figure 12. Expense overview screen interface.

Figure 12 illustrates the Expense Overview and Expense by Category screens. The screen displays a graph of total expenses for the current month and previous months, with the total expenses for the current month grouped by category below, and a comparison of increases and decreases compared to previous months.

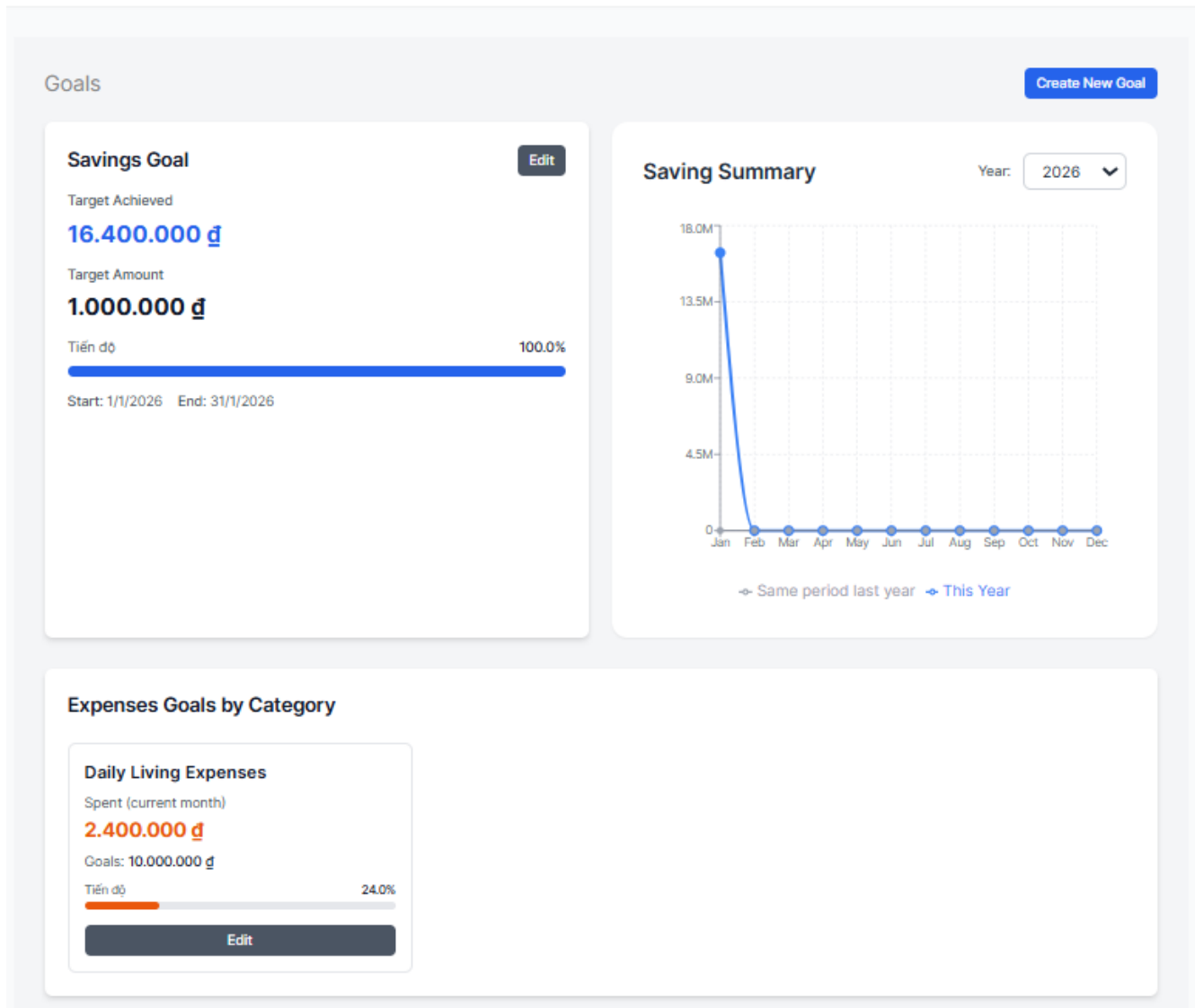


Figure 13. Interface for viewing monthly goals.

Figure 13 illustrates the Monthly Goal View screen. The screen displays savings goals, amounts achieved, and comparison rates. Users can view and edit savings and spending goals by category. The Savings Overview chart allows users to compare net savings by month in the current year and the previous year, helping to assess financial improvement over time.

The screenshot shows a web application interface for 'Personal Financial Management'. The top navigation bar includes links for Dashboard, Account, Transactions, Expenses, Bills, Goals, and a user profile for Tran Hoang Viet with a Log out button. The main content area is titled 'Goals' and features a 'Create New Goal' button. A 'Savings Goal' card is visible, showing a target achieved of 16,400,000 đ and a target amount of 1,000,000 đ, with a progress bar at 100.0%. A 'Saving Summary' chart for the year 2026 is also present. In the foreground, a modal titled 'Add a new goal' is open, containing a form with the following fields: 'Goal Type' with radio buttons for 'Overall savings' (selected) and 'Spending limits'; 'Target amount' with a text input field; 'Start date' and 'End date' with date pickers showing the format 'mm/dd/yyyy'. The modal has 'Cancel' and 'Save' buttons at the bottom right.

Figure 14. Add target screen interface.

Figure 14 illustrates the Add Goal screen. The screen displays a form for entering information such as Goal Type, Amount, Start Date, and End Date. The user clicks Save to add the new goal to the system.

Bills

LOGO	ITEM DESCRIPTION	DUE DATE	LAST CHARGE	AMOUNT	ACTION
C	Clothes	07/01/2026	07/01/2026	2.000.000 đ	Pay Now
E	Electric power bill	08/01/2026	08/01/2026	515.000 đ	Pay Now
W	Water bill	09/01/2026	09/01/2026	95.000 đ	Pay Now

Figure 15. Interface for viewing the list of invoices.

Figure 15 illustrates the "View Upcoming Invoices" screen. The screen displays invoices with due dates after the current date, the item name, the nearest renewal date, and the amount due.

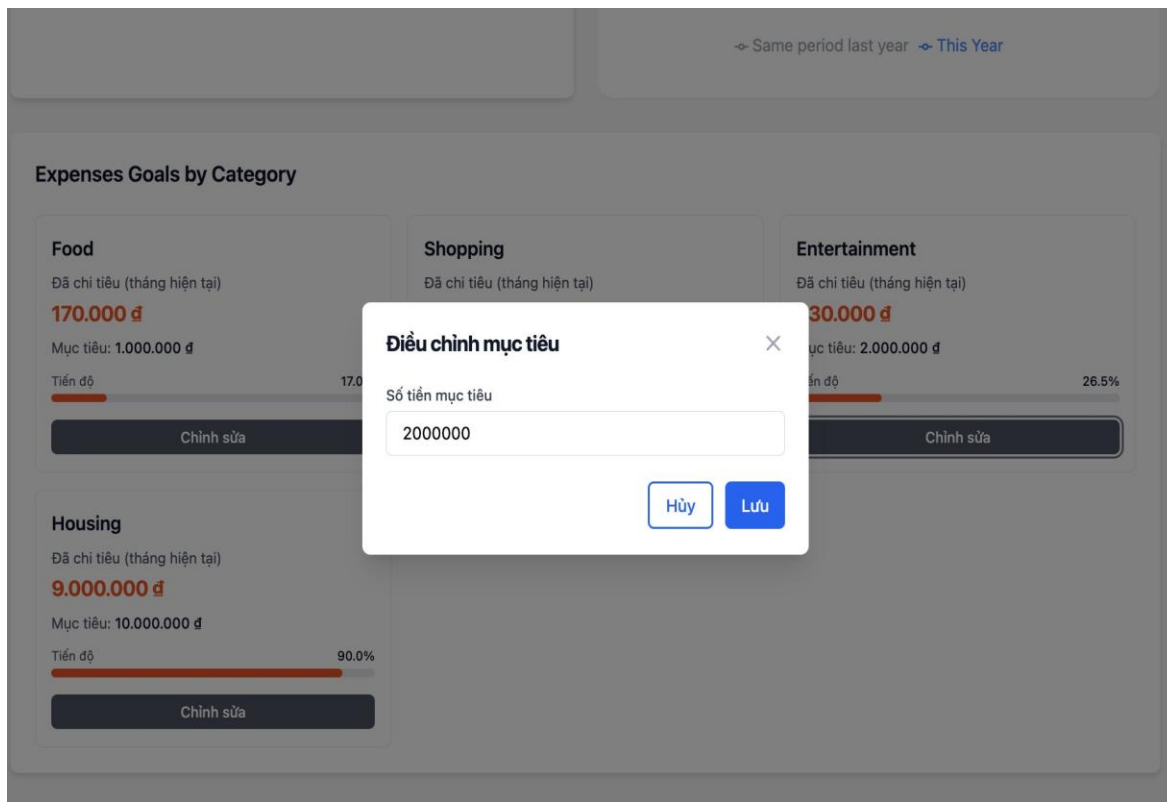


Figure 16. Target editing screen interface.

Figure 16 illustrates the Add Goal screen. The screen displays a form for entering information such as Goal Type, Amount, Start Date, and End Date. The user clicks Save to add the new goal to the system.

Source code and automatically generated files

After completing the experimental process, the entire source code of the system was compiled and published at the following link (https://github.com/vietth2004/VC-AWG-Demo_FinalCode) to facilitate readers' reference and verification of the results .

List of newly generated files in the Backend

Table 17 lists all newly created Backend files along with their purpose, creation/modification status, and corresponding use cases.

Table 17. List of folders containing files created in the Backend

File	Purpose	Related use case
modules/auth/	Login/Registration API	Login, Register
modules/account/	API for performing CRUD operations for account	View/Add/Edit/Delete for account
modules/transaction/	API for transactions	View transaction history, Create transactions
modules/bill/	API for bills	View the list of bills
modules/category/	API for categories	See the category list
modules/expenses/	Expense analysis API	View monthly expenses, detailed by category
modules/goal/	API performs CRUD operations for the target.	View, Create, Edit Items
modules/savings/	API aggregation for cost savings	View the overall chart Savings

Figure 17 illustrates the results of the automatically generated source code in the Backend directory:

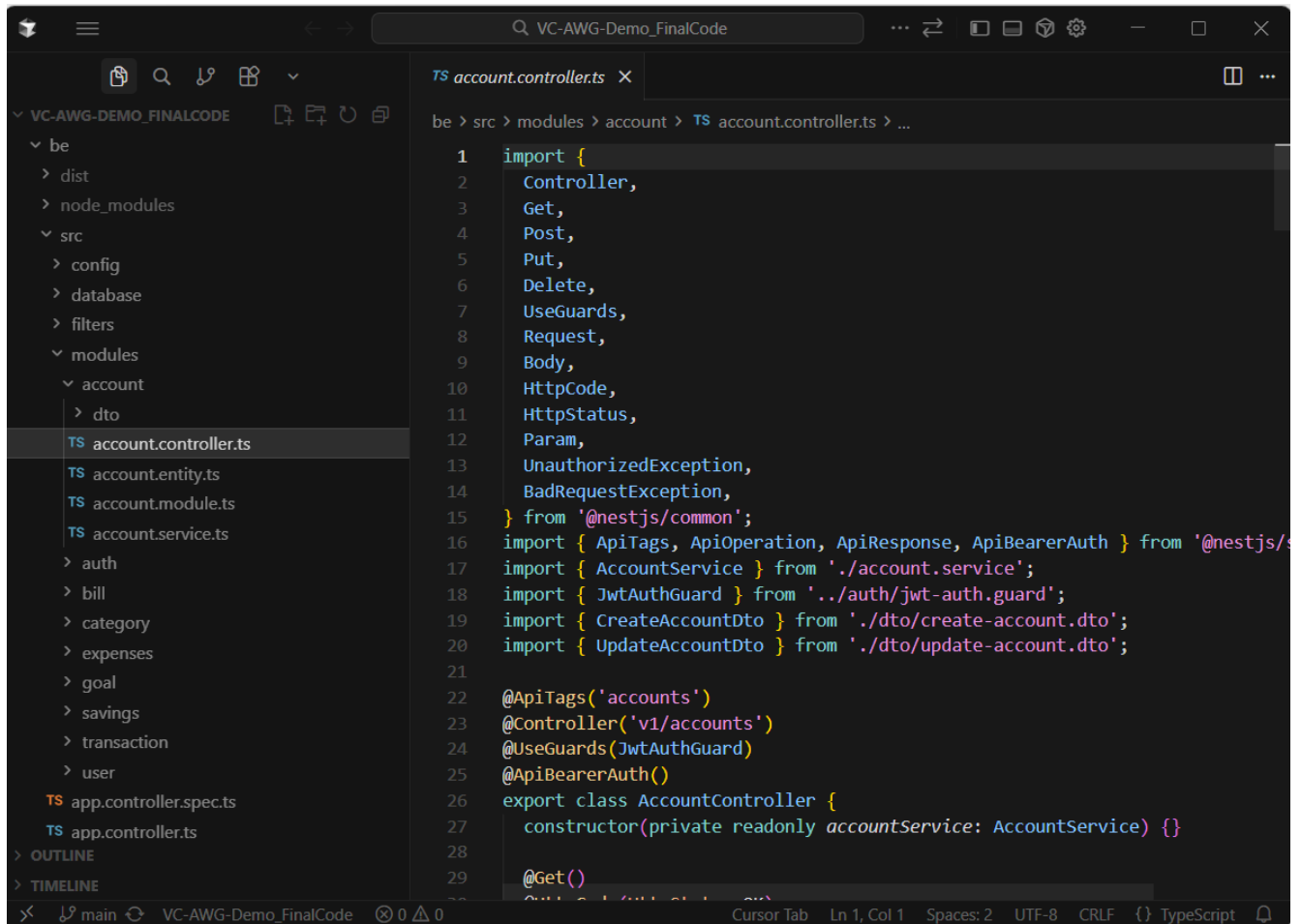


Figure 17. The generated source code in the Backend folder.

List of newly generated directories in Frontend

Table 18 presents a list of newly created Frontend files, describing the function of each file and its relationship to the system's use cases.

Table 18. List of new directories and files created in the Frontend

File	Purpose	Related use cases
LoginForm/LoginForm.tsx	Login form	Log in
SignUpForm/SignUpForm.tsx	Registration form	Register
AddTransactionForm/ AddTransactionForm.tsx	Add transaction form	Create a new transaction
AddAccountForm/ AddAccountForm.tsx	Add account form	Add account

AccountEditForm/ AccountEditForm.tsx	Account editing form	Edit account
DeleteAccountModel/ DeleteAccountModal.tsx	Modal deletes account	Delete account
UpcomingBills/ UpcomingBills.tsx	Upcoming invoice display	View the list of invoices
ExpensesBreakdown/ ExpensesBreakdown.tsx	Expenditure details by category	View spending details
ExpenseSummaryChart/ ExpenseSummaryChart.tsx	Monthly spending chart	View inventory spending month
SavingsSummaryChart/ SavingsSummaryChart.tsx	Savings chart	View savings chart
CreateGoalModal/ CreateGoalModal.tsx	Goal creation modal	Set new goals
AdjustGoalModal/ AdjustGoalModal.tsx	Modal for adjusting targets	Adjust the target
Toast/Toast.tsx	Display quick (toast) notifications	The whole website
AddTransaction/ AddTransactionPage.tsx	Add transaction page	Create a new transaction
AccountList/ AccountListPage.tsx	Account list page	View the list of assets
AccountDetail/ AccountDetailPage.tsx	Account details page	View account details
AddAccount/ AddAccountPage.tsx	Add account page	Add account
Expenses/ExpensesPage.tsx	Expense analysis page	View monthly expenses, detailed by category.

Figure 18 illustrates the results of the automatically generated source code in the Frontend directory:

```
1 import React, { useEffect, useState } from 'react'
2 import { useAuth } from '../../../context/AuthContext'
3 import { accountService } from '../../../api/account.service'
4 import { transactionService } from '../../../api/transaction.service'
5 import { Account, Transaction } from '../../../api/types'
6 import Loading from '../../../components/Loading/Loading'
7 import Error from '../../../components/Error/Error'
8
9 const Dashboard: React.FC = () => {
10   const { user } = useAuth()
11   const [accounts, setAccounts] = useState<Account[]>([])
12   const [recentTransactions, setRecentTransactions] = useState<Transaction[]>([])
13   const [isLoading, setIsLoading] = useState(true)
14   const [error, setError] = useState('')
15
16   useEffect(() => {
17     const fetchData = async () => {
18       try {
19         setIsLoading(true)
20         const [accountsRes, transactionsRes] = await Promise.all([
21           accountService.getAccounts(),
22           transactionService.getTransactions({
23             type: 'All',
24             limit: 5
25           })
26         ])
27
28         if (accountsRes.success && accountsRes.data) {
29           // Hỗ trợ nhiều kiểu response shape từ backend
30         }
31       } catch (error) {
32         setError(error.message)
33       }
34     }
35     fetchData()
36   })
37 }
```

Figure 18. The generated source code in the Frontend folder.